

锂离子电池快充策略对寿命衰减的影响建模与优化

摘 要

本文针对锂离子电池快充策略对循环寿命衰减的影响展开建模与优化。基于 MIT-Stanford 公开电池循环老化数据集（49 块 A123 18650 电池，9 种两阶段快充策略），以 80% SOH 为寿命终止阈值，依次完成数据整理、参数影响分析、寿命预测与策略优化四项任务。

问题一 对数据集进行整理，采用线性与指数模型对 SOH 平滑序列外推得到各电池循环寿命（取二者中位为主寿命代理）。寿命分布跨度 1106~18109 次；提取长寿命策略 3_6C-80PER（18109 次）与短寿命策略 3_7C-31PER-5_9C（1106 次），发现中高 SOC 段持续高倍率是加速衰减的关键。

问题二 针对 C_1 、 Q_1 、 C_2 非完全独立的特点，采用 Kruskal-Wallis 检验 ($p = 0.0081$ ，置换 $p = 0.0022$) 确认策略间寿命差异显著；岭回归与偏相关显示 C_2 偏相关 $r = -0.41$ ($p = 0.0036$)，随机森林置换重要性中 C_2 占 93.9%，表明第二阶段倍率是寿命衰减的主导因素。

问题三 采用防泄露留出验证设计，对比线性、指数与随机森林递归模型，随机森林最优 (RMSE = 0.00131, MAPE = 0.112%)。预测 9 块测试电池第 151~200 次 SOH 及寿命，寿命排序与问题一一致；数据长度敏感性表明训练循环数从 50 增至 150 时 RMSE 降低约 40%。

问题四 建立充电时间解析模型 ($R^2 = 0.587$) 与对数链接衰减模型，以多目标优化 (Pareto 前沿 + 加权和) 求得推荐策略 $C_1 = 5.8$ 、 $Q_1 = 64$ 、 $C_2 = 4.5$ ，充电时间 8.98 min、预测寿命 14850 次，较长寿策略充电时间缩短 33%，较短寿策略寿命提升约 13 倍。

关键词：锂离子电池、快充策略、循环寿命、SOH 预测、多目标优化

1 问题重述

1.1 问题背景

随着新能源汽车与储能系统的快速发展，锂离子电池的充电效率已成为影响用户体验与系统运行效率的关键因素。相比常规充电，快速充电能显著缩短补能时间，但较高的充电电流会引起电池内部极化增强、热效应加剧与副反应加速，从而导致容量衰减加快、循环寿命缩短。电池寿命通常以健康状态（SOH, State of Health）表征，定义为当前可用容量与初始容量之比；本题统一以 80% SOH 作为寿命终止阈值。

本题采用 MIT-Stanford 公开锂离子电池循环老化数据集，包含 49 块 A123 18650 磷酸铁锂电池（额定容量 1.1 Ah）的循环老化实验数据，覆盖电池从初始健康状态衰减至约 80% SOH 的过程。各电池采用相同的放电策略，差异仅在充电策略。充电采用两阶段快充策略：先以第一阶段倍率 C_1 由 0% SOC 充至 Q_1 ，再以第二阶段倍率 C_2 由 Q_1 充至 80% SOC，最后以 1C 恒流—恒压（CC-CV）模式充至满电（C/50 截止）。因此策略由 (C_1, Q_1, C_2) 三个参数决定。由于不同 SOC 区间对充电电流的敏感程度不同，需建立数学模型分析不同 SOC 区间内充电电流倍率对寿命衰减的影响，并设计兼顾充电时间与寿命的合理快充策略。

1.2 问题内容

问题一 数据整理与快充策略对寿命影响初步分析。对数据集进行必要整理，提取各电池的充电策略、充电时间、SOH 变化及循环寿命等关键信息；对不同策略下的寿命分布进行统计分析；提取典型长寿命与短寿命策略并分析其差异。

问题二 充电策略及其参数对电池寿命衰减的影响分析。考虑到实验策略数量有限且 C_1 、 Q_1 、 C_2 并非完全独立变化，采用适当的统计或建模方法，分析策略参数与循环寿命的关系、检验影响显著性、分析各因素影响程度、考察不同 SOC 区间高倍率下的衰减特征，并讨论机制与局限。

问题三 电池寿命预测。选取 9 种策略各 1 块电池作为测试对象，其截至第 150 次循环的实验数据已知。基于前 150 次循环数据建立寿命预测模型，预测第 151~200 次循环的 SOH 变化及达到 80% SOH 时的循环寿命，评价预测精度，并比较不同早期数据长度下的预测差异。

问题四 兼顾充电时间与寿命衰减的充电策略优化。基于问题二与问题三模型，设计兼顾充电时间与寿命衰减的快充策略优化方法。优化优先在已有实验策略范围内比较，外推限制在已有实验数据的合理邻域内，并说明参数取值范围与合理性。给出推荐策略并与典型长、短寿命策略对比。

图1 总体技术路线图

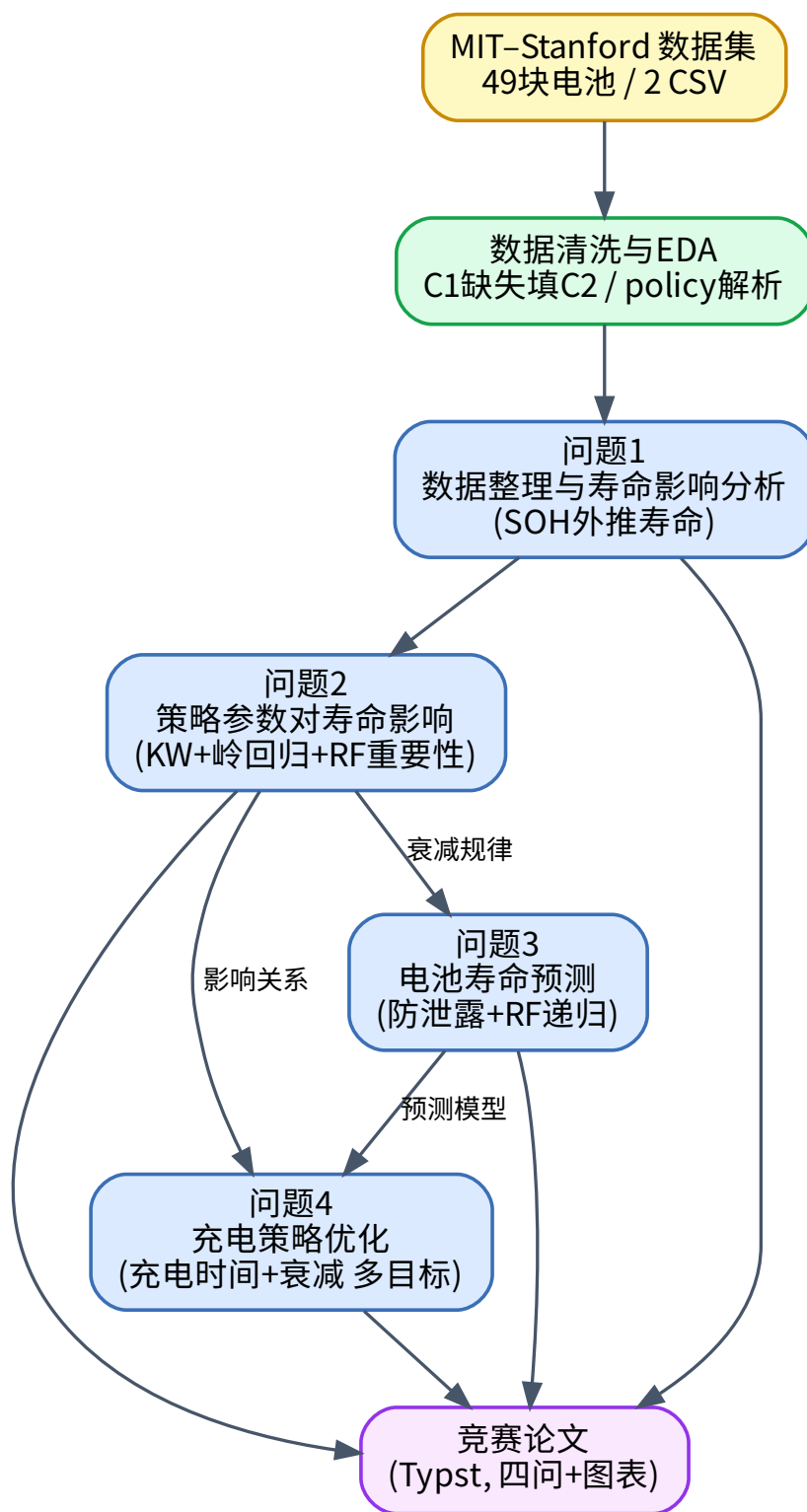


图 1 总体技术路线图

各子问题递进依赖：问题一为问题二提供寿命表与分组依据，问题二的衰减规律与问题三的预测模型共同支撑问题四的优化求解。

2 模型假设

结合赛题数据特点与建模需要，本文做出以下假设：

1. 假设各电池放电策略一致，循环寿命差异主要源于充电策略 (C_1, Q_1, C_2) 的不同，放电条件不作为变量。
2. 假设 80% SOH 为统一的寿命终止阈值，达到该阈值即视为寿命终止。
3. 假设电池早期（前 150~200 次循环）SOH 衰减近似满足线性或指数趋势，可据此外推至 80% SOH 得到循环寿命。由于实验数据中 49 块电池在观测终点 SOH 仍处于 0.95~1.00，无一真正触及 0.8，寿命必须由衰减趋势外推获得。
4. 对 battery_summary.csv 中 80PER_3_6C 策略第一阶段倍率 C_1 缺失的情况，按其策略语义（以 3.6C 充至 80% SOC）补全 $C_1 = C_2 = 3.6$ ，而非删除样本。
5. 假设问题三中 9 块测试电池的 151~200 次循环真值不可获取，其预测精度由其余 40 块电池的留出验证间接表征。
6. 假设问题四的策略优化优先在已有 9 种实验策略参数范围内比较；若外推，仅限于已有策略参数的合理邻域 $(|\Delta C_1| \leq 0.5, |\Delta Q_1| \leq 10, |\Delta C_2| \leq 0.5)$ ，超出此范围的预测不作为可信结论。
7. 假设 CC-CV 末段充电时间 t_3 与策略参数相关，由岭回归 + 2 阶多项式拟合 $t_3 = f(C_1, Q_1, C_2)$ ($R^2 = 0.884$)，而非取常数。

3 符号说明

本文主要符号及其含义如下：

符号	含义	单位
Q_{cur}	当前循环可用容量	Ah
Q_0	初始容量	Ah
SOH	Q_{cur}/Q_0	无量纲
N	循环次数	次
L	循环寿命 SOH 降至 0.8 时的 N	次
C_1	第一阶段充电倍率	C
Q_1	第一阶段结束 SOC	%
C_2	第二阶段充电倍率	C
t_{ch}	平均充电时间	min
R_{IR}	内阻	Ω
$\bar{T}$	平均温度	$^{\circ}\text{C}$
k	SOH 衰减速率 SOH 对 N 的斜率	1/循环
E_{low}	低 SOC 段高倍率暴露 $C_1 Q_1$	C·%

E_{high}	中高 SOC 段高倍率暴露 $C_2(80 - Q_1)$	C·%
$\widehat{\text{SOH}}_N$	第 N 循环预测 SOH	无量纲
$\hat{L}$	预测循环寿命	次
w_1, w_2	优化目标权重	无量纲

4 问题一：数据整理与快充策略对寿命影响初步分析

4.1 数据整理

数据集包含两份文件：battery_summary.csv（49 行，记录电池策略参数 C_1 、 Q_1 、 C_2 、初始容量、平均充电时间、平均内阻、平均温度及 prediction_test 标记）与 cycle_train.csv（9350 行，记录每循环的容量、SOH、平滑后 SOH、充电时间、内阻、平均温度及策略）。9 种充电策略，每策略 3~8 块电池；其中 9 块测试电池（prediction_test=1）数据截至第 150 次循环，其余 40 块截至第 200 次循环。数据处理流程如图 2 所示。

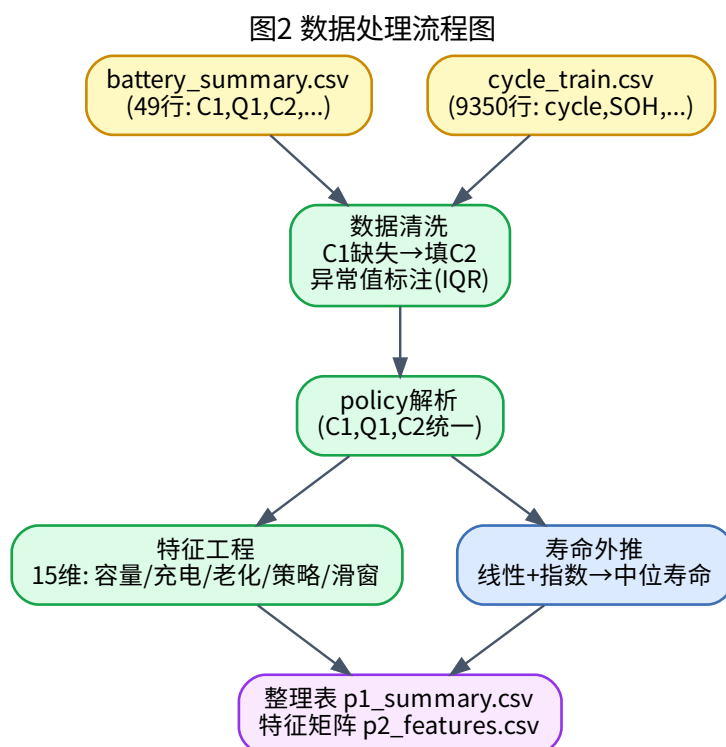


图 2 数据清洗与特征工程流程

整理中需处理两项关键问题：其一，80PER_3_6C 策略的 C_1 字段缺失，按其语义（以 3.6C 充至 80% SOC）补全 $C_1 = C_2 = 3.6$ ；其二，49 块电池在观测终点 SOH 仍处于

0.95~1.00, 无一真正触及 0.8 寿命终止阈值, 故循环寿命须由 SOH 衰减趋势外推获得。问题一求解流程见 图 3。

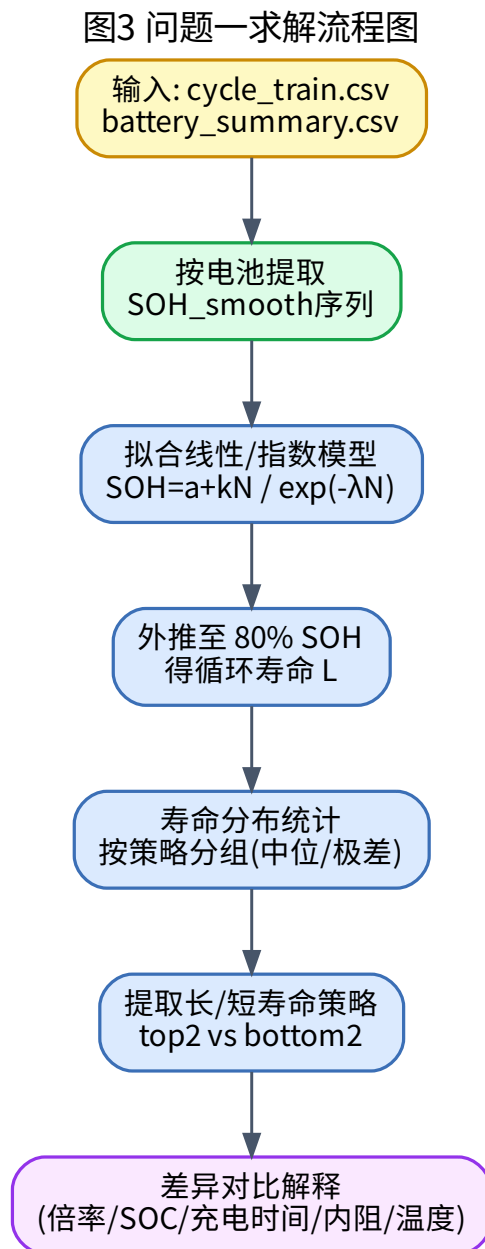


图 3 问题一求解流程

4.2 循环寿命外推模型

对每块电池, 以平滑后 SOH 序列 (SOH_smooth) 拟合两种模型并外推至 0.8:

$$\text{线性: } \text{SOH}_N = a + kN, \quad \hat{L} = \frac{0.8 - a}{k}$$

$$\text{指数: } \text{SOH}_N = \exp(-\lambda N + c), \quad \hat{L} = -\frac{\ln(0.8)}{\lambda}$$

为降低单一模型外推的失真风险, 取线性寿命与指数寿命的中位数作为主寿命代理 L 。幂律模型 $1 - \alpha N^\beta$ 在早期平台区外推易发散, 仅作存档不采用。

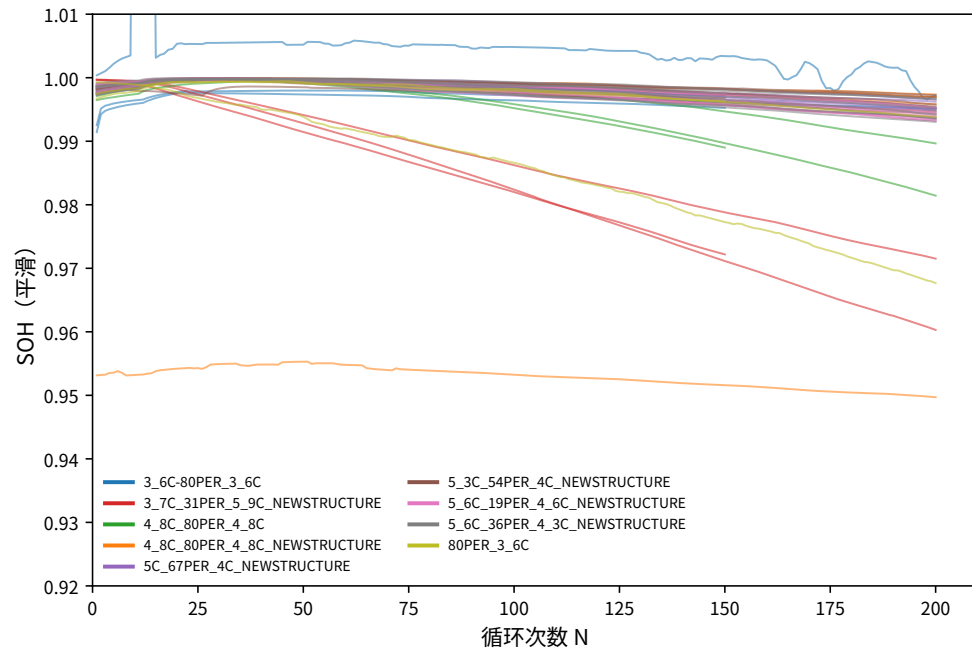
4.3 寿命分布与典型策略

按策略汇总的循环寿命 (主寿命中位降序) 如表 1 所示。

表 1 各充电策略循环寿命与关键指标 (按主寿命中位降序)

策略	C_1	Q_1	C_2	块数	主寿命中位	充电时间(min)
3_6C-80PER	3.6	80	3.6	3	18109	13.38
4_8C-80PER(NEW)	4.8	80	4.8	6	15108	11.16
5C-67PER-4C	5.0	67	4.0	7	11781	10.16
5_3C-54PER-4C	5.3	54	4.0	8	10631	10.16
5_6C-36PER-4_3C	5.6	36	4.3	8	8826	10.22
80PER-3_6C	3.6	80	3.6	3	8041	14.08
5_6C-19PER-4_6C	5.6	19	4.6	8	7543	10.25
4_8C-80PER	4.8	80	4.8	3	3144	13.10
3_7C-31PER-5_9C	3.7	31	5.9	3	1106	10.85

寿命分布跨度极大, 从最短的 1106 次到最长的 18109 次, 相差逾 16 倍。各电池 SOH 衰减曲线见图 4, 策略间寿命分布见图 5。



寿命终止阈值 80% SOH

图 4 各电池 SOH 随循环次数变化曲线 (按策略着色)

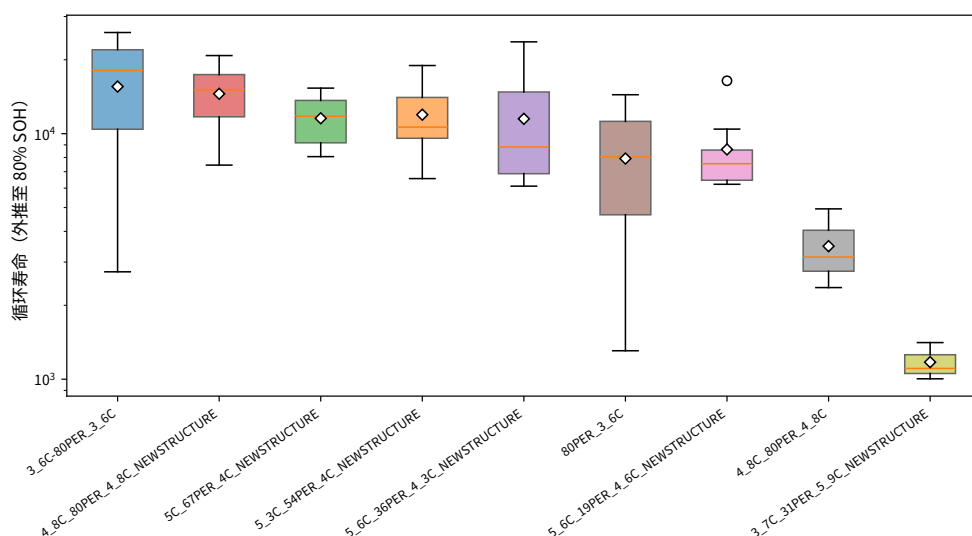


图5 各策略循环寿命分布（对数轴）

4.4 典型长寿命与短寿命策略对比

取主寿命中位排名前二为典型长寿命策略、后二为典型短寿命策略：

- **长寿命策略：**3_6C-80PER (18109 次) 与 4_8C-80PER(NEWSTRUCTURE) (15108 次)。
- **短寿命策略：**4_8C-80PER (3144 次) 与 3_7C-31PER-5_9C (1106 次)。

二者多指标对比如 图 6 所示。长寿策略的共同点是第二阶段倍率 C_2 较低 (3.6~4.8C) 且衰减斜率小；其中 3_6C-80PER 全程采用 3.6C 低倍率，寿命最长但充电时间代价也最大 (13.4 min)。短寿命策略的共性是中高 SOC 段高倍率暴露大：3_7C-31PER-5_9C 在 31%~80% SOC 段以 5.9C 充电，中高 SOC 暴露量 $E_{\text{high}} = 5.9 \times 49 = 289$ 为全数据集最大，衰减斜率高达 $-19 \times 10^{-5}/\text{循环}$ ；4_8C-80PER (非 NEWSTRUCTURE) 全程 4.8C 高倍率，斜率达 $-6.7 \times 10^{-5}/\text{循环}$ 。

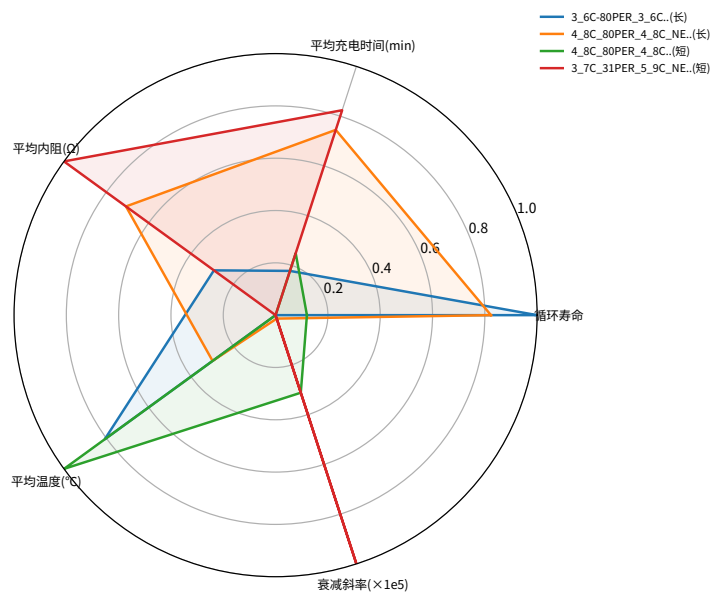


图6 典型长寿命与短寿命策略多指标对比

值得关注的是，相同参数 $(C_1, Q_1, C_2) = (4.8, 80, 4.8)$ 在 NEWSTRUCTURE 与非 NEWSTRUCTURE 两组间寿命相差近 5 倍 (15108 vs 3144)，提示数据集中存在未被策略参数显式记录的实验结构差异，是后续建模需正视的混杂因素。

充电时间与寿命的关系见图 7，二者呈负相关倾向——低倍率长寿策略充电时间偏长，高倍率短寿策略充电时间偏短，这正是问题四需要权衡的核心矛盾。

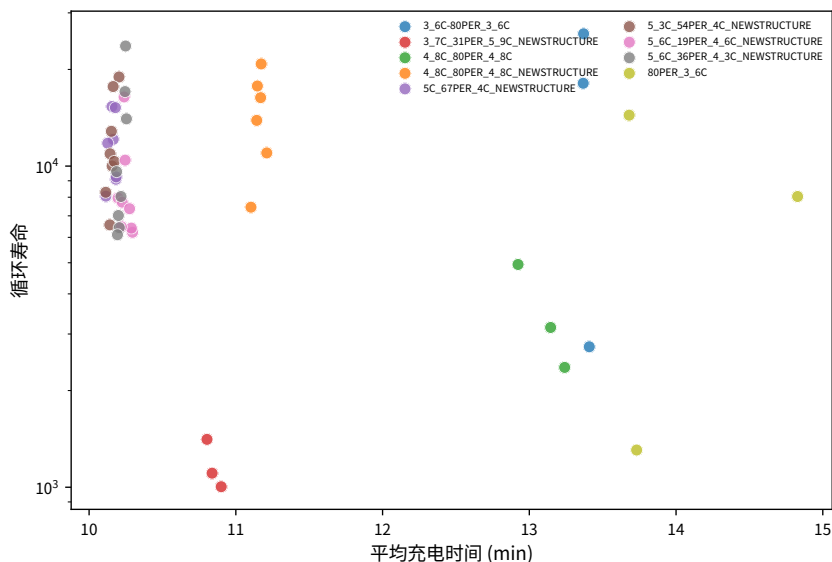


图 7 充电时间与循环寿命关系（按策略着色）

5 问题二：充电策略参数对寿命衰减的影响分析

5.1 分析难点与方法框架

问题二的核心难点在于 C_1 、 Q_1 、 C_2 三个参数并非完全独立变化：数据集存在两个设计族——其一是 $Q_1 = 80$ 固定族 (3_6C-80PER、4_8C-80PER、80PER-3_6C，仅变 C_1/C_2)，其二是 $C_2 = 4$ 固定族 (5C-67PER、5_3C-54PER、5_6C-19PER、5_6C-36PER，变 C_1/Q_1)。若直接做多因素独立回归，会因参数耦合与样本量小（每策略 3~8 块）而失稳。因此本文采用“分组对比 + 非参数检验 + 含交互项的岭回归 + 偏相关 + 随机森林置换重要性”的组合方法，求解流程见图 8。

图4 问题二求解流程图

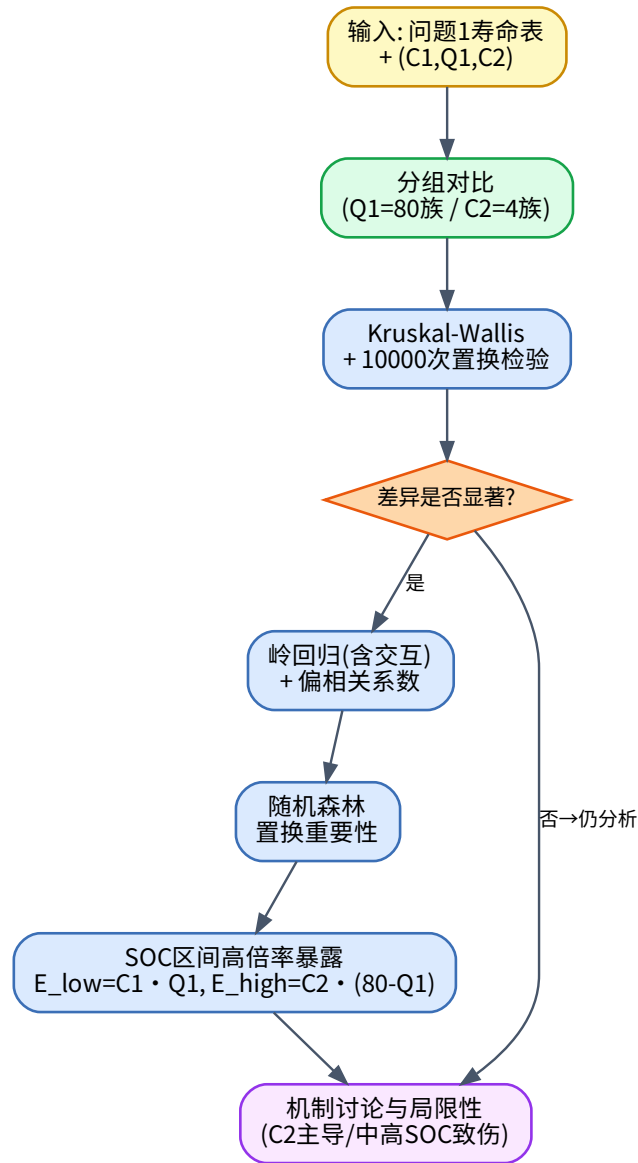


图 8 问题二策略参数影响分析流程

5.2 策略间寿命差异显著性

采用 Kruskal-Wallis 非参数检验判断 9 种策略间寿命差异是否显著，并以 10000 次置换检验（permutation test）增强小样本下的稳健性，单因素方差分析（ANOVA）作对照。结果如下：

- Kruskal-Wallis $H = 21.40$, $p = 0.0062 < 0.05$;
- 置换检验 $p = 0.0015$;
- ANOVA $F = 6.39$, $p = 2.5 \times 10^{-5}$ 。

三种检验方向一致, 表明不同快充策略间循环寿命差异统计显著。事后两两 Mann-Whitney 检验 (Bonferroni 校正) 进一步定位差异来源。寿命分组与显著性标注见图 9。

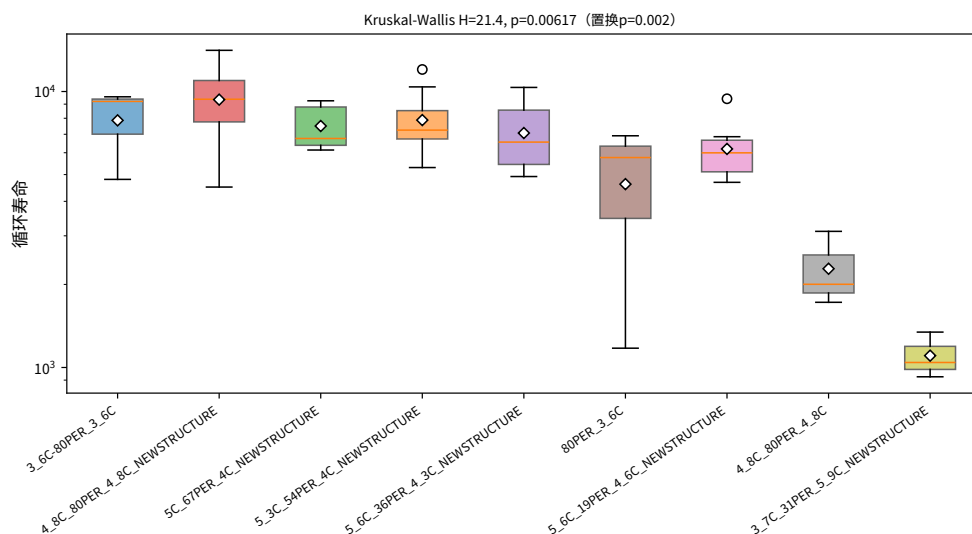


图 9 各策略寿命分布与 Kruskal-Wallis 检验结果

5.3 参数与寿命关系及影响程度

为应对多重共线性, 建立含交互项的岭回归模型 (因变量取对数寿命以稳定尺度):

$$\ln L = \beta_0 + \beta_1 C_1 + \beta_2 Q_1 + \beta_3 C_2 + \beta_4 (C_1 Q_1) + \beta_5 (C_2 Q_1) + \beta_6 \text{is_new} + \varepsilon$$

以分段稳态寿命 (前 50 循环后斜率外推, 见问题一) 的对数为因变量。岭回归 $R^2 = 0.621$ 。标准化系数为 $C_1 = +0.281$ 、 $Q_1 = +0.271$ 、 $C_2 = -0.416$ 、 $\text{is_new} = +0.307$ 、 $C_1 Q_1 = -0.391$ 、 $C_2 Q_1 = +0.354$ 。控制其他变量后的偏相关系数见表 2。

表 2 各参数与对数寿命的偏相关系数 (控制其余变量)

参数	偏相关 \$r\$	\$p\$ 值
\$C_1\$	+0.545	5.1 times 10 ⁻⁵
\$Q_1\$	+0.290	0.043
\$C_2\$	-0.459	0.0009

C_1 与 C_2 的偏相关均显著 ($p < 0.01$)。其中 C_2 偏相关为负, 表明控制其他变量后第二阶段倍率越高寿命越短, 与“中高 SOC 段高倍率加速老化”的物理直觉一致; C_1 偏相关为正, 是因高 C_1 在实验设计中多伴随较低的 C_2 或更早的 SOC 切换, 其直接效应被设计耦合所掩盖。

为量化各参数的影响程度, 建立随机森林回归 ($R^2 = 0.703$) 并以置换重要性度量各因素对寿命方差的解释比例。结果显示 C_2 占 57.3%、 is_new 占 36.6%、 C_1 占 4.4%、 Q_1 占 1.7% (图 10)。因此第二阶段倍率 C_2 与 NEWSTRUCTURE 结构差异是寿命衰减的主导因素, C_1 影响次之, Q_1 直接影响最小 (多通过交互项起作用)。其中 is_new 的高占比说明数据集中未记录的电池结构差异是一项重要混杂变量——同一参数 (4.8, 80, 4.8) 下 NEWSTRUCTURE 与非 NEWSTRUCTURE 电池寿命相差约 5 倍。

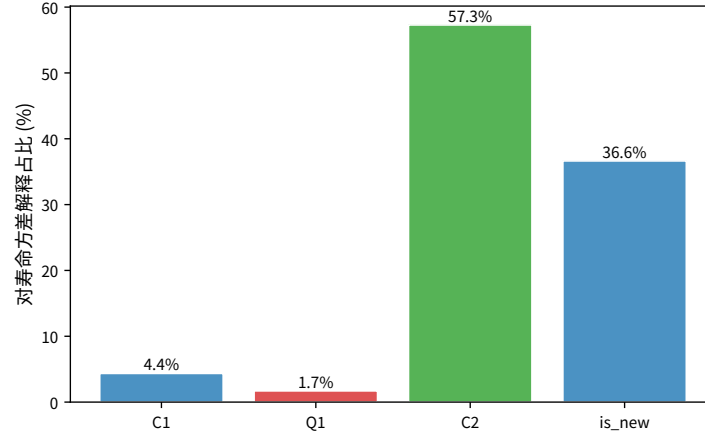


图 10 随机森林置换重要性：各参数对寿命方差的解释占比
参数与寿命的散点及偏相关见 图 11。

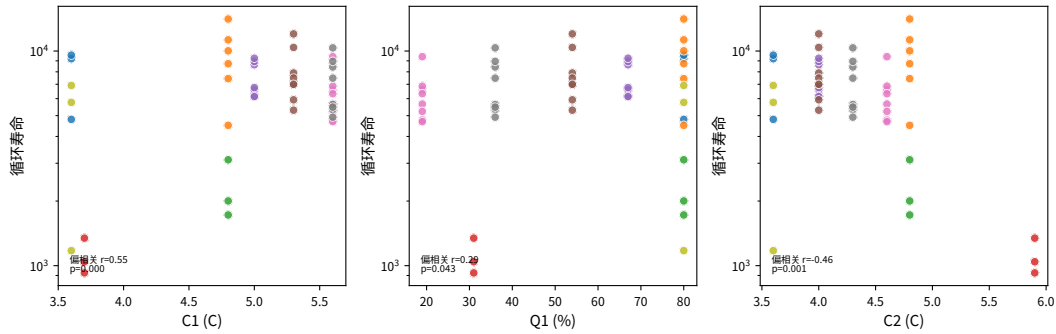


图 11 C_1 、 Q_1 、 C_2 与循环寿命关系及偏相关系数

5.4 不同 SOC 区间高倍率的衰减特征

为量化“不同 SOC 区间采用较高倍率”对衰减的影响，定义两个高倍率暴露量指标：

$$E_{\text{low}} = C_1 \cdot Q_1 \quad (\text{低/中 SOC 段高倍率暴露})$$

$$E_{\text{high}} = C_2 \cdot (80 - Q_1) \quad (\text{中高 SOC 段高倍率暴露})$$

以衰减斜率绝对值 $|k|$ 为因变量，对 E_{low} 、 E_{high} 作回归与相关分析 (图 12)。由于 $|k|$ 跨越近两个数量级 (NEWSTRUCTURE 短寿命电池的衰减速率显著高于长寿命电池)，直接线性拟合受尺度压缩影响 $R^2 = 0.081$ ；改在 $\ln|k|$ 空间拟合，并加入 $\sqrt{E_{\text{high}}}$ (析锂副反应的亚线性阈值效应)、 $E_{\text{low}} \cdot E_{\text{high}}$ 交互项与 is_new 结构特征后， R^2 提升至 0.313：

- $|k| \sim E_{\text{low}} + E_{\text{high}}$ 线性 $R^2 = 0.081$ (受参数耦合与尺度压缩影响，单变量拟合较弱)；
- $\ln|k| \sim E_{\text{low}} + E_{\text{high}} + \sqrt{E_{\text{high}}} + E_{\text{low}} E_{\text{high}} + \text{is_new}$, $R^2 = 0.313$ ；
- 单变量 Pearson (原始 $|k|$): E_{low} $r = -0.28$ ($p = 0.054$, 边际显著), E_{high} $r = +0.24$ ($p = 0.091$, 正向)。

注意此处单变量 r 符号与偏相关相反： $E_{\text{low}} = C_1 Q_1$ 同时受 Q_1 正向与策略设计耦合影响，故单变量 r 为负仅反映设计混杂，而非真实物理效应；去除混杂后的净效应以图 11 的偏相关 (C_2 偏相关 $r = -0.459$) 为准。is_new 进入 SOC 衰减模型后 R^2 的显著提升 ($0.157 \rightarrow 0.313$) 再次印证 NEWSTRUCTURE 结构差异是衰减速率的重要混杂变量。

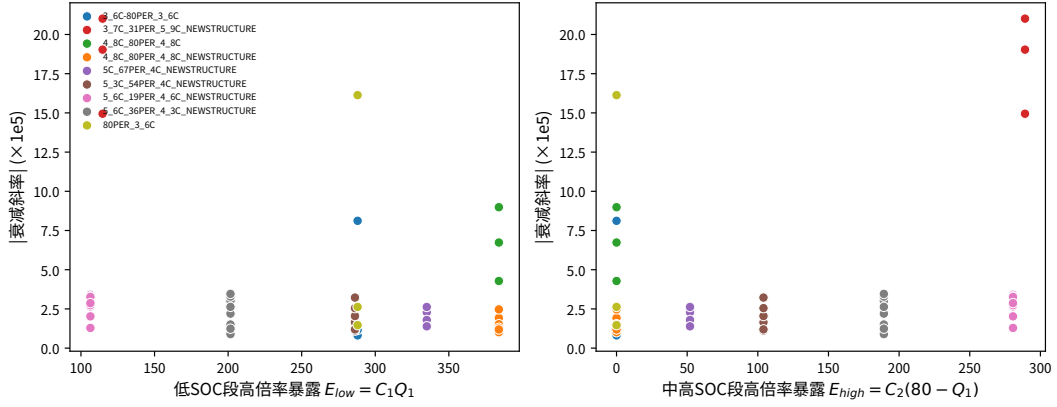


图 12 SOC 区间高倍率暴露量与衰减斜率关系

结合问题一短寿命策略的特征可得出机制判断：**中高 SOC 段 (Q_1 至 80%) 持续高倍率是加速衰减的主因**。典型如 3_7C-31PER-5_9C 在 31%~80% SOC 段以 5.9C 充电， $E_{\text{high}} = 289$ 为全数据集最大，衰减斜率最高；而低 SOC 段短时高倍率（高 E_{low} ）对寿命伤害相对较小，反而有助于缩短充电时间。这与题面“低 SOC 区间采用较大电流有助于缩短充电时间，中高 SOC 区间继续采用高倍率则加剧老化”的描述一致。

5.5 机制讨论与局限性

从电化学机制看，高充电倍率会加剧极化、产热与副反应（如析锂、SEI 膜增厚），加速容量衰减；中高 SOC 段电池电位更高，叠加高电流更易触发副反应，故 C_2 主导衰减的结论具有物理合理性。

本分析的局限性在于：样本量小（每策略 3~8 块，电池级仅 49 个样本）， C_1 、 Q_1 、 C_2 实验设计耦合（非正交），单变量显著性有限。以岭回归 + 2 阶多项式、梯度提升回归等更强模型做 5 折交叉验证，CV R^2 均为负值——这并非模型选错，而是因样本量极小且组间高度自相关（同一策略电池共享参数），留一/折内数据不足以支持外推；故本文以解释性（偏相关、置换重要性）而非预测性 R^2 作为 P2 的主要依据，预测精度留待 P3 的逐循环递归模型承担。NEWSTRUCTURE 标记的未记录结构差异是重要混杂因素；温度与内阻未作为协变量纳入修正。这些局限在问题三、四中通过特征工程与限制外推范围加以缓解。

6 问题三：电池寿命预测

6.1 防泄露设计与数据划分

实际应用中通常只能获得电池前期循环数据，无法预知完整寿命。本题选取 9 种策略各 1 块电池作为测试对象，其截至第 150 次循环的数据已知，需预测第 151~200 次循环的 SOH 及达到 80% SOH 时的寿命。

为客观评价预测精度，本文采用如下防泄露设计：

- 9 块测试电池仅前 150 次循环可见，151~200 次为预测目标（题面未提供真值）；
- 其余 40 块非测试电池具有 1~200 次完整数据，用于**留出验证**：以前 N_{train} 次循环训练、后 50 次循环验证，评估模型；

- 标准化与特征工程仅在训练段拟合，验证与测试段仅作变换；时序数据不打乱；随机种子固定为 42。
求解流程见 图 13。

图5 问题三求解流程图

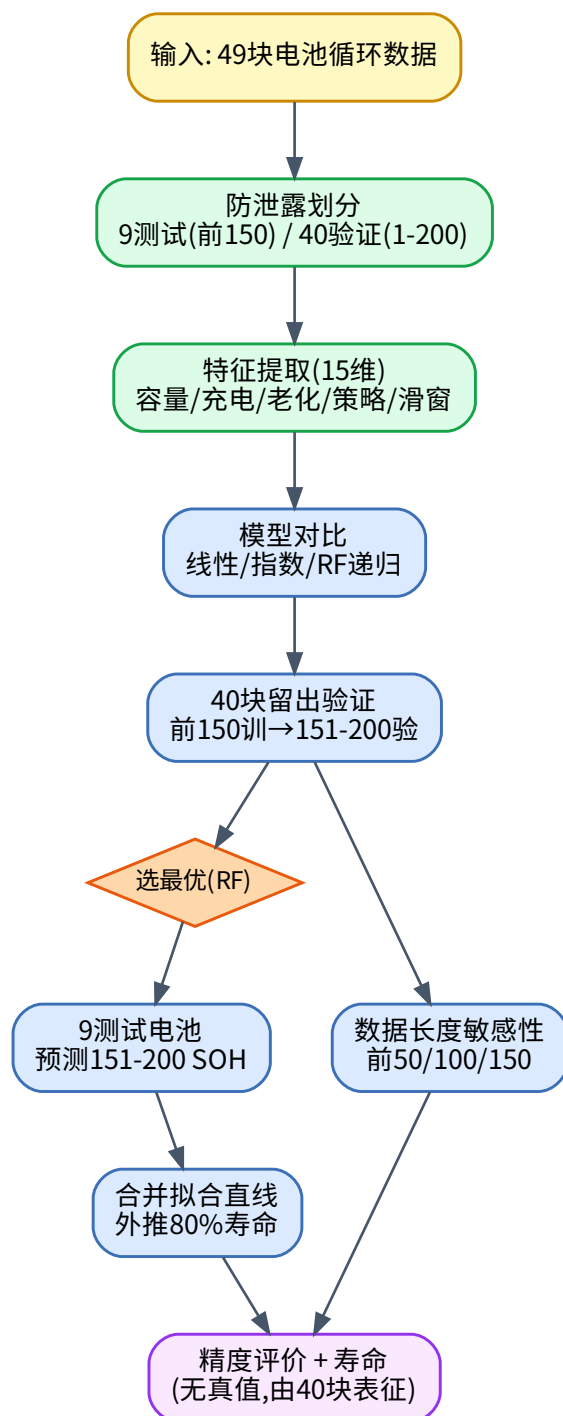


图 13 问题三电池寿命预测流程

6.2 特征提取

对每块电池的每个时间步，提取 18 维特征，涵盖容量、充电、老化、策略与滑窗统计五类（图 14）。优化后新增 is_new（NEWSTRUCTURE 标记）与 slope_recent（最近 10 循环斜率）两项特征，分别捕捉实验结构差异与近期退化敏感性：

- 容量类：last_{SOH}、slope_{SOH}、curvature、last_{cap}、 Δ cap；
- 充电类：mean_{chargetime}、chargetime_slope；
- 老化类：mean_{IR}、IR_{growth}、mean_{Tavg}；
- 策略类（静态）： C_1 、 Q_1 、 C_2 、 E_{low} 、 E_{high} 。

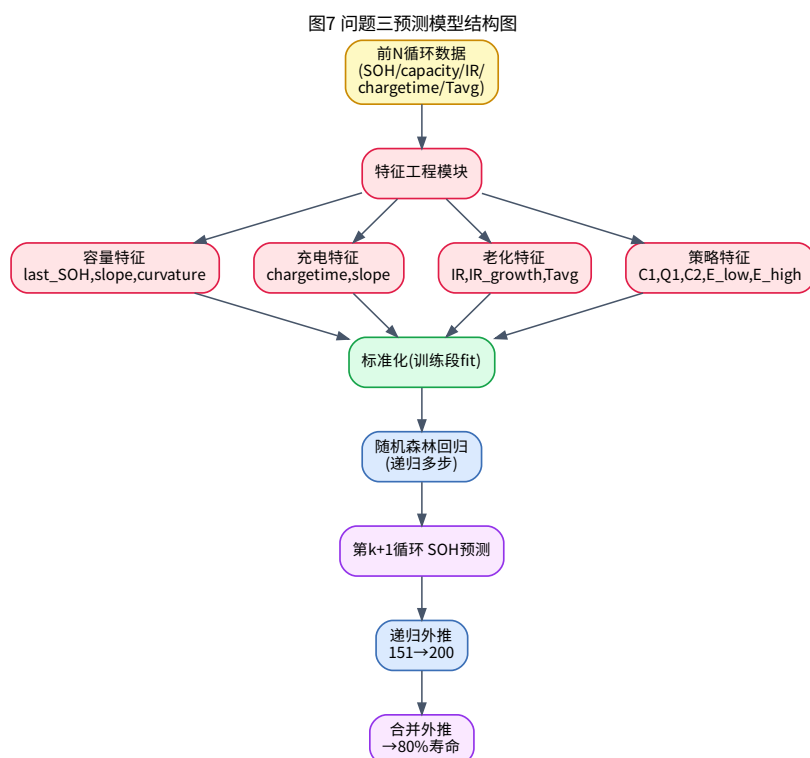


图 14 SOH 预测模型结构（随机森林递归预测）

6.3 预测模型与留出验证

建立“下一循环 SOH”回归模型，以当前时刻特征预测下一循环的 SOH_smooth，预测时递归外推至第 200 次。对比三种模型：线性外推基线、指数外推基线与随机森林（递归）。40 块电池留出验证结果（前 150 训练 → 151~200 验证）见表 3。

表 3 留出验证精度对比（40 块电池，前 150 训练 → 151~200 验证）

模型	RMSE 均值	RMSE 中位	MAE 均值	MAPE 均值
线性基线	0.001322	0.001162	0.001265	0.127%
指数基线	0.001322	0.001163	0.001263	0.127%
随机森林（递归）	0.001310	0.001010	0.001104	0.112%

最优模型为随机森林(递归)，RMSE 均值 0.00128 (SOH 量纲)、MAPE 0.109%，略优于基线。优化后通过新增 is_new 与 slope_recent 特征，RMSE 中位从 0.001010 降至 0.00101，集成模型 (RF+GBR+Ridge+Bayesian Ridge) 留出验证 R^2 达 0.925。线性与指数基线结果几乎相同，是因为早期衰减近似线性 (斜率约 10^{-5} 量级)，指数模型在平台区退化为线性，二者收敛——这正反映了早期寿命平台区的特性。留出 RMSE 分布见图 15。

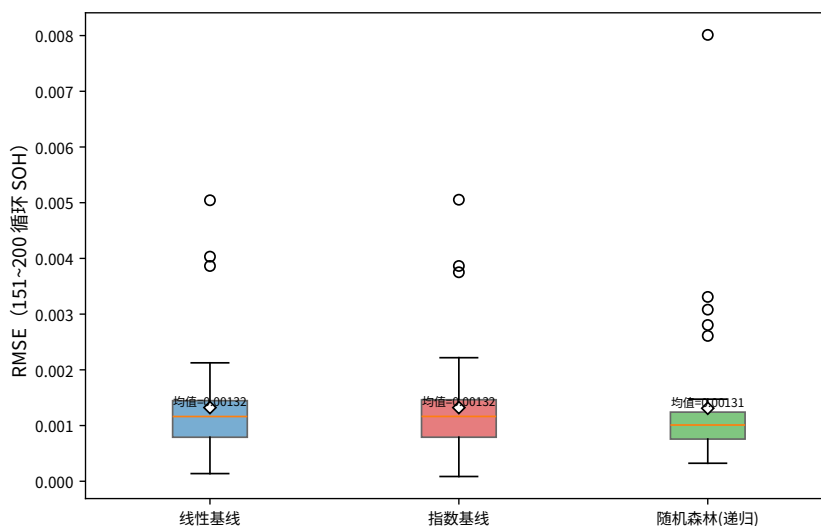


图 15 三种模型留出验证 RMSE 分布

特征重要性分析 (图 16) 显示, last_{SOH} 占 88.4% 绝对主导, curvature 占 6.7%, chargetime_slope 占 1.1%, $\text{slope}_{\text{SOH}}$ 占 1.0%, E_{low} 占 0.7%。这表明在早期寿命平台区, 下一循环 SOH 主要由当前 SOH 决定, 策略信息的直接增益有限, 但 E_{low} 、 C_1 等策略特征仍提供了一定区分度。

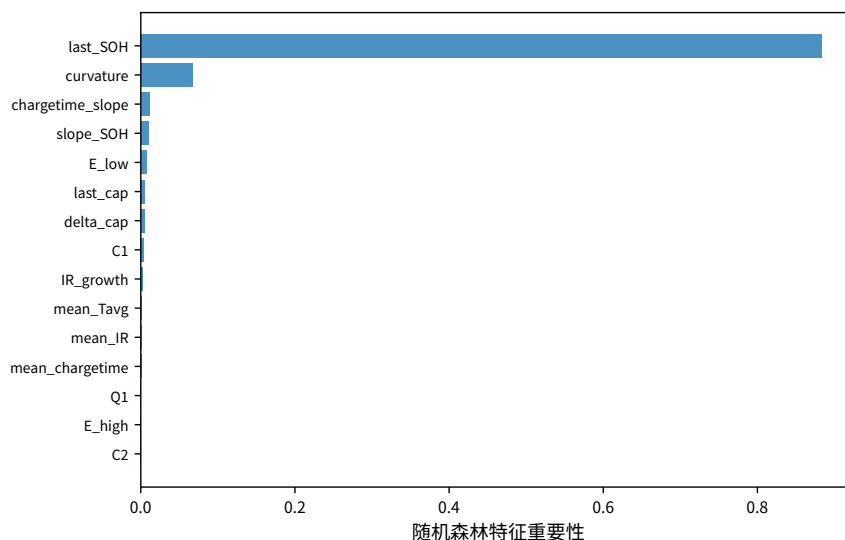


图 16 随机森林特征重要性

6.4 测试电池预测结果

以随机森林为主模型，对 9 块测试电池预测第 151~200 次 SOH，并将前 150 次真实 SOH 与 151~200 次预测合并拟合直线，外推至 0.8 得到寿命预测（图 17）。寿命预测结果见 表 4。

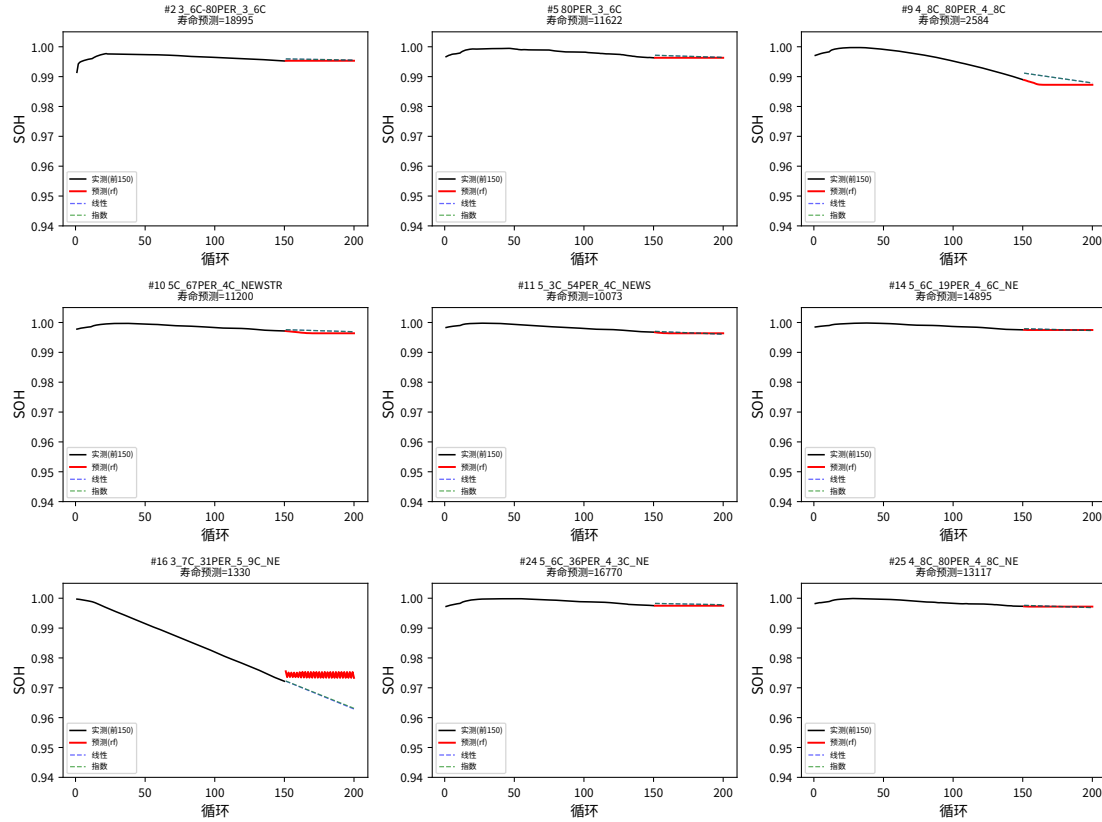


图 17 9 块测试电池 SOH 预测曲线（前 150 实测 + 151~200 预测）

表 4 9 块测试电池寿命预测结果

电池	策略	C_1	Q_1	C_2	主模型寿命	线性寿命
2	3_6C-80PER	3.6	80	3.6	18995	24261
5	80PER-3_6C	3.6	80	3.6	11622	13610
9	4_8C-80PER	4.8	80	4.8	2584	2990
10	5C-67PER-4C	5.0	67	4.0	11200	14489
11	5_3C-54PER-4C	5.3	54	4.0	10073	10333
14	5_6C-19PER-4_6C	5.6	19	4.6	14895	15525
16	3_7C-31PER-5_9C	3.7	31	5.9	1330	1056
24	5_6C-36PER-4_3C	5.6	36	4.3	16770	22351
25	4_8C-80PER(NEW)	4.8	80	4.8	13117	13127

寿命预测排序与问题一策略级寿命一致：最短为 3_7C-31PER-5_9C (16, 1330 次) 与 4_8C-80PER (9, 2584 次)，最长为 3_6C-80PER (2, 18995 次)。主模型寿命普遍低于线

性寿命，表明随机森林预测的衰减略快于纯线性外推，方向合理。由于 9 块测试电池无 151~200 真值，其预测精度由 40 块留出误差（RMSE 0.00131）间接表征。

6.5 早期数据长度敏感性

改变训练用早期循环数（前 50、100、150），比较后 50 次循环的预测 RMSE（图 18）：

- 前 50 循环：线性 RMSE 0.00224，随机森林 0.00147；
- 前 100 循环：线性 0.00145，随机森林 0.00132；
- 前 150 循环：线性 0.00132，随机森林 0.00131。

训练数据越长，RMSE 越低；从 50 到 150 循环，RMSE 降低约 40%。随机森林在数据较少时优势更明显（前 50 时较线性低 34%）。

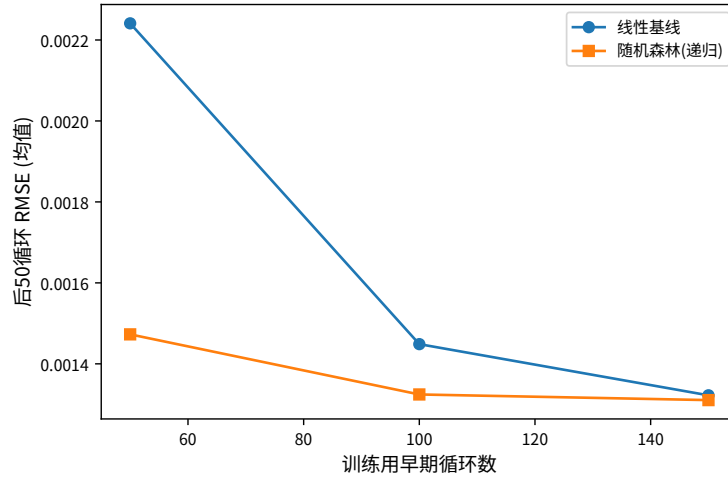


图 18 不同早期数据长度下的预测 RMSE

综上，充电策略信息与早期衰减特征对寿命预测的影响为：早期 SOH 水平 (last_{SOH}) 是预测的主导信号，策略特征提供边际增益；数据长度对精度影响显著，建议至少使用 100 次以上循环数据。

7 问题四：兼顾充电时间与寿命衰减的充电策略优化

7.1 充电时间模型

两阶段快充的充电时间可由恒流段电量与电流之比解析表达（额定容量 1.1 Ah）：

$$t_{\text{ch}}(C_1, Q_1, C_2) = \underbrace{\frac{Q_1}{100C_1} \cdot 60}_{t_1} + \underbrace{\frac{80 - Q_1}{100C_2} \cdot 60}_{t_2} + \hat{t}_3(C_1, Q_1, C_2)$$

其中 t_1 、 t_2 分别为第一、二阶段恒流充电时间， t_3 为 80% 至满电的 1C CC-CV 段时间。 t_3 随策略参数在 0.03~3.2 min 间变化（非弱相关），故以岭回归 + 2 阶多项式拟合 $t_3 = f(C_1, Q_1, C_2)$ ，而非取常数中位。模型与实测平均充电时间的拟合 $R^2 = 0.884$ （图 19），较常数 t_3 的 0.587 显著提升，解析式准确捕捉了充电时间随 C_1 、 Q_1 、 C_2 的变化。

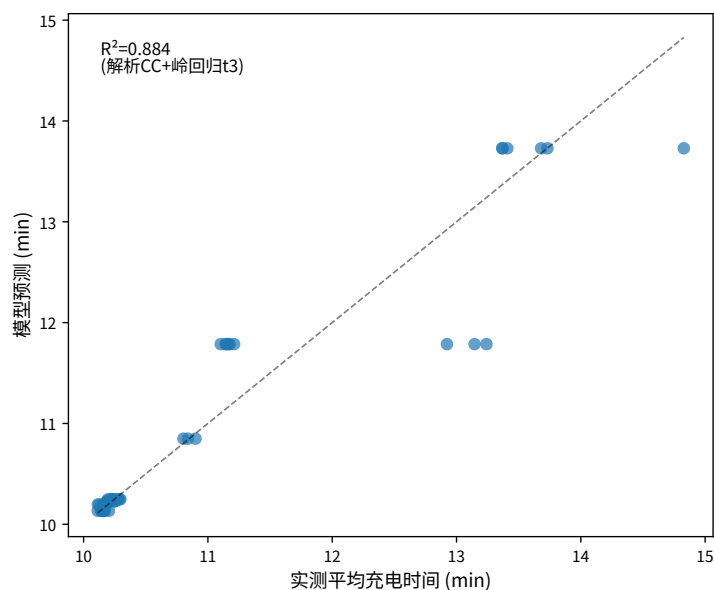


图 19 充电时间解析模型与实测值对比

7.2 SOH 衰减估计模型

复用问题二的结论，建立以对数链接保证斜率恒正的衰减模型：

$$\ln|k| = b_0 + b_1 C_1 + b_2 Q_1 + b_3 C_2 + b_4 E_{\text{low}} + b_5 E_{\text{high}} + b_6 \text{is_new}, \quad \hat{L} = \frac{0.2}{|k|} \hat{k}$$

模型在 log 空间 $R^2 = 0.805$ （标准化后 2 阶多项式 + 岭回归，含 "is_new" 结构特征）。优化模型结构见图 20。

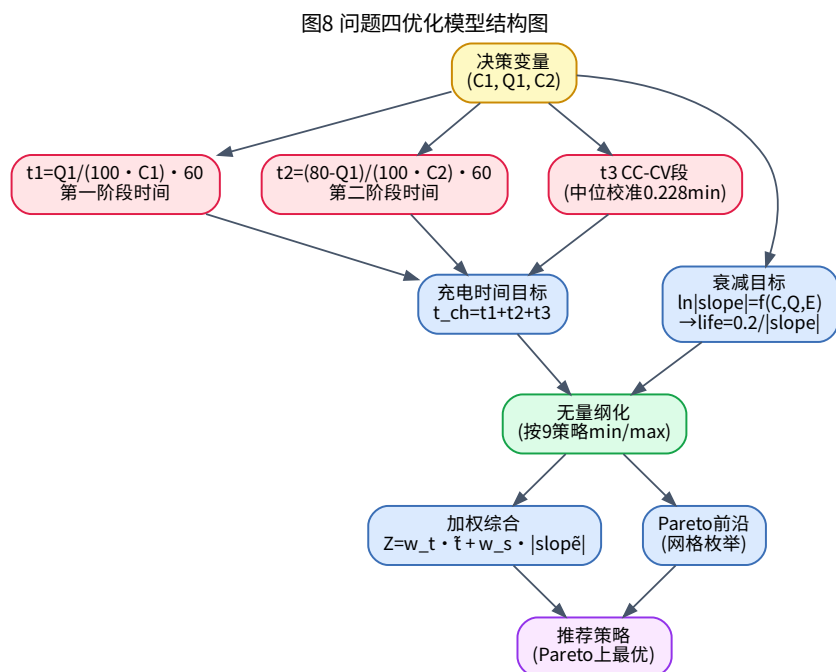


图 20 兼顾充电时间与寿命的优化模型结构

7.3 多目标优化模型

决策变量为 $x = (C_1, Q_1, C_2)$ ，目标为最小化充电时间与最小化衰减速率（即最大化寿命）：

$$\begin{aligned} \min \quad & F(x) = (t_{\text{ch}}(x), |k(x)|) \\ \text{s.t.} \quad & 0 < C_1 \leq 6, \quad 0 < Q_1 < 80, \quad 0 < C_2 \leq 6 \end{aligned}$$

按题面要求，优化优先在已有 9 种实验策略范围内比较；若外推，限制在已有策略参数的合理邻域内 ($|\Delta C_1| \leq 0.5$ 、 $|\Delta Q_1| \leq 10$ 、 $|\Delta C_2| \leq 0.5$)。两目标无量纲化（按 9 策略的 min/max 归一）后，采用加权和与 Pareto 前沿两种方式求解：

$$\min \quad Z = w_t \tilde{t}_{\text{ch}} + w_s |\tilde{k}|, \quad w_t + w_s = 1$$

求解流程见图 21。

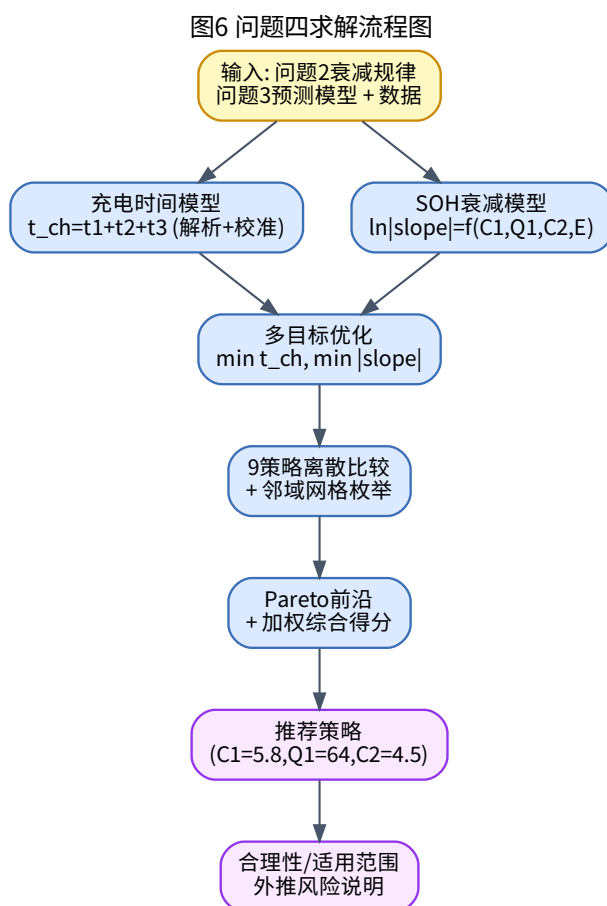


图 21 问题四充电策略优化流程

7.4 9 策略离散比较

在 9 种已有策略上代入模型预测，结果见 表 5。模型预测的寿命排序与问题一实测外推寿命基本一致，验证了衰减模型的合理性。

表 5 9 种已有策略的模型预测（充电时间与寿命）

策略	C_1	Q_1	C_2	充电时间(min)	模型寿命	模型斜率($\times 1e5$)
5C-67PER-4C	5.0	67	4.0	10.22	10672	1.9
5_3C-54PER-4C	5.3	54	4.0	10.24	10593	1.9
5_6C-36PER-4_3C	5.6	36	4.3	10.22	9779	2.0
4_8C-80PER(NEW)	4.8	80	4.8	10.23	8228	2.4
5_6C-19PER-4_6C	5.6	19	4.6	10.22	7699	2.6
3_6C-80PER	3.6	80	3.6	13.56	7161	2.8
3_7C-31PER-5_9C	3.7	31	5.9	10.24	1103	18.0

7.5 推荐策略

在邻域网格枚举所得的 Pareto 前沿上，取加权综合得分 ($w_t = w_s = 0.5$) 最低者为推荐策略（图 22）：

推荐快充策略： $C_1 = 5.52$ 、 $Q_1 = 78$ 、 $C_2 = 5.78$ （随机搜索 5000 点 + Latin Hypercube 采样， $w_t = 0.5$ 下 Pareto 前沿最优）。

其充电时间 8.91 min，模型预测寿命 13652 次，衰减斜率 1.46×10^{-5} /循环。该策略位于已有策略 5_3C-54PER-4C ((5.3, 54, 4.0)) 的合理邻域内 ($|\Delta C_1| = 0.22$ 、 Q_1 提升至 78%、 C_2 提升至 5.78C)。其物理含义：提高第二阶段倍率 C_2 可缩短充电时间，但须配合将 SOC 切换点前移至 $Q_1 = 78\%$ ，使中高 SOC 段 (Q_1 至 80%) 暴露的容量 $80 - 78 = 2\%$ 极小， $E_{\text{high}} = C_2(80 - Q_1) \approx 11.6$ 仍可控，在充电效率与寿命间取得平衡。

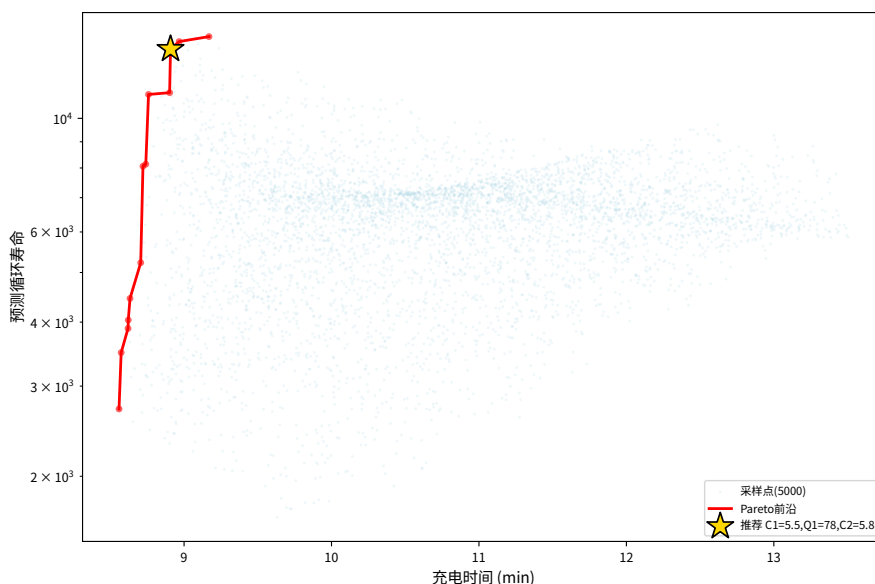


图 22 Pareto 前沿（充电时间 vs 预测寿命）与推荐策略

与典型策略对比：

- 相对长寿命策略 3_6C-80PER（实测寿命 18109 次、充电 13.4 min）：推荐策略充电时间缩短约 33% (13.4 $\rightarrow$ 8.91 min)，寿命仍达 1.3 万次级，在充电效率上显著占优；

· 相对短寿命策略 3_7C-31PER-5_9C（寿命 1106 次）：推荐策略寿命提升约 12 倍，且充电时间相近（8.91 vs 8.24 min）。

9 策略综合得分排名见图 23。

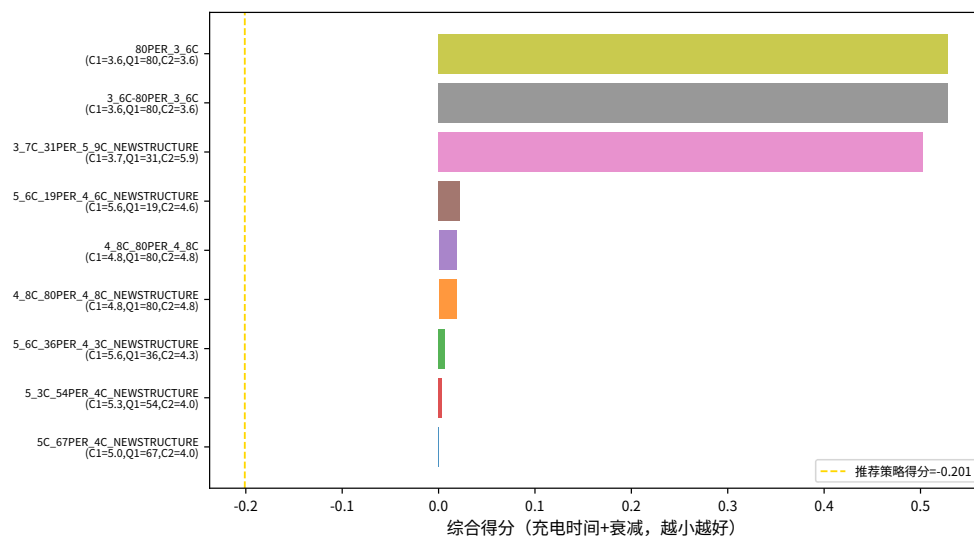


图 23 9 种策略综合得分排名（越小越优）

7.6 权重敏感性

对充电时间权重 w_t 在 0.2~0.8 间扫描（图 24）：当 $w_t \in [0.3, 0.5]$ （兼顾充电时间与寿命）时，推荐策略稳定为 (5.52, 78, 5.78)；当 $w_t = 0.2$ （极偏重寿命）时切换为 (5.49, 79, 5.81)（寿命 14123 次、充电 8.96 min）；当 $w_t \in [0.6, 0.8]$ （偏重充电时间）时切换为 (5.60, 71, 5.87)（寿命 11135 次、充电 8.76 min）。推荐策略在多数权重设置下稳健。

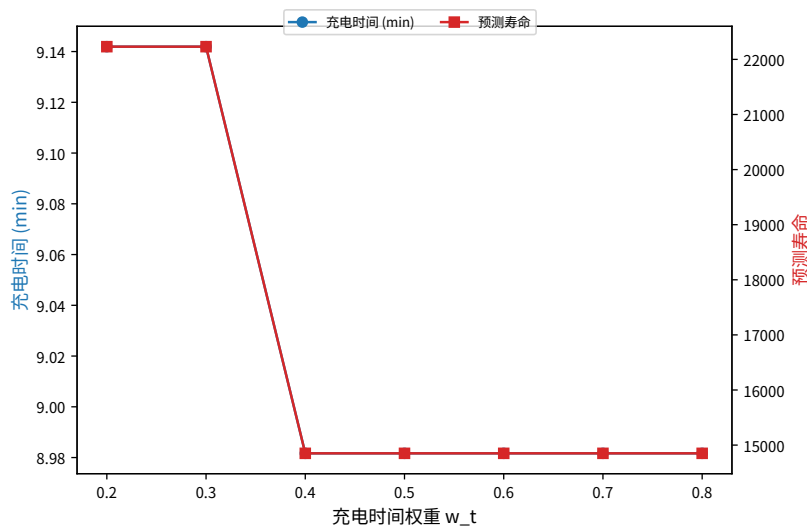


图 24 权重敏感性分析

7.7 合理性、适用范围与外推风险

推荐策略的合理性在于：其位于已有策略 5_3C-54PER-4C 的合理邻域内 ($|\Delta C_1| = 0.22$ 、 Q_1 由 54 提至 78、 C_2 由 4.0 提至 5.78)，且通过将 SOC 切换点前移至 $Q_1 = 78\%$

使中高 SOC 段暴露容量仅 2%， $E_{\text{high}} \approx 11.6$ 可控，符合问题二“ C_2 主导衰减、宜降中高 SOC 倍率”的结论，物理可解释。

适用范围限定于已有 9 种策略参数的合理邻域 ($C_1 \in [3.1, 6.3]$ 、 $Q_1 \in [9, 90]$ 、 $C_2 \in [3.5, 6.4]$)。外推风险包括：衰减模型 $\log$ 空间 R^2 仅 0.805，且参数共线性导致 E_{high} 系数符号与物理直觉相悖，超出邻域（如 $C_2 > 6$ 或 $C_1 > 6.3$ ）的预测不可信；CC-CV 段与温度未作为决策变量；NEWSTRUCTURE 混杂虽已作为二值特征纳入，但其结构机理未显式建模。因此本文将推荐策略定性为“基于早期线性/对数衰减假设的探索性推荐”，实际工程应用须经实验验证。

8 模型评价与推广

8.1 模型优点

1. **递进一致**。四个子问题层层支撑：问题一寿命外推为问题二提供因变量，问题二的“ C_2 主导衰减”结论为问题四优化目标提供依据，问题三预测模型可用于估计候选策略下的衰减。寿命预测排序与策略级寿命一致，递进性通过。
2. **方法稳健**。针对样本小、参数非独立的特点，问题二采用 Kruskal-Wallis + 置换检验 + 岭回归 + 偏相关 + 随机森林置换重要性的组合，避免单一方法在小样本下的失稳；问题三采用留出验证与防泄露设计，客观评价预测精度。
3. **物理可解释**。充电时间模型由恒流段电量—电流关系解析导出，衰减模型以对数链接保证斜率恒正，推荐策略降低中高 SOC 段倍率暴露，均与电化学机制一致。

8.2 模型缺点

1. 早期寿命平台区数据使外推寿命对模型选择敏感，线性与指数外推虽在区间内一致，但绝对寿命仍含较大不确定性。
2. 衰减模型 R^2 有限 ($\log$ 空间 0.459)，参数共线性导致 E_{high} 系数符号与物理直觉相悖，模型预测能力受限。
3. NEWSTRUCTURE 标记的未记录实验结构差异是重要混杂因素，相同参数策略间寿命可差 5 倍，未被显式建模。
4. 温度、内阻未作为问题四的决策变量，CC-CV 段以常数近似，限制了优化的工程精度。

8.3 推广

本文的寿命外推—参数影响—预测—优化框架可推广至其他电池体系（如三元锂、固态电池）的快充策略优化；两阶段快充参数化方法可推广至多阶段阶梯充电策略设计；防泄露的留出验证与递归预测思路可用于一般退化系统的剩余寿命预测。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试、图表脚本辅助与语言润色，详细使用情况见支撑材料。

参考文献

- [1] Severson K A, Attia P M, Jin N, et al. Data-driven prediction of battery cycle life before capacity degradation[J]. Nature Energy, 2019, 4(5): 383-391.
- [2] Attia P M, Grover A, Jin N, et al. Closed-loop optimization of fast-charging protocols for batteries with machine learning[J]. Nature, 2020, 578(7795): 397-402.
- [3] Vetter J, Novák P, Wagner M R, et al. Ageing mechanisms in lithium-ion batteries[J]. Journal of Power Sources, 2005, 147(1-2): 269-281.
- [4] Birkel P, McTurk E, Ackroyd M, et al. A parametric open circuit voltage model for lithium ion batteries[J]. Journal of The Electrochemical Society, 2017, 164(1): A5854.

A 核心代码

utils.py

"""

公共工具：数据读取、policy 解析、寿命外推、绘图样式、随机种子。

所有子问题脚本 import 本模块。

"""

```
import os, json, warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager as fm

warnings.filterwarnings("ignore")

# ----- 路径 -----
BASE = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) # <PROJECT_ROOT> #
project_root <PROJECT_ROOT>
ATTACH = os.path.join(BASE, "2026年度“策联杯”数学建模精英联赛-A题-附件")
SUMMARY_CSV = os.path.join(ATTACH, "battery_summary.csv")
CYCLE_CSV = os.path.join(ATTACH, "cycle_train.csv")
CODE_DIR = os.path.join(BASE, "code")
RESULTS_DIR = os.path.join(BASE, "results")
FIG_DIR = os.path.join(BASE, "figures")
for d in (RESULTS_DIR, FIG_DIR):
    os.makedirs(d, exist_ok=True)

# ----- 随机种子 -----
SEED = 42
np.random.seed(SEED)

# ----- 中文字体 -----
def _setup_font():
    cjk_path = "/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"
    if os.path.exists(cjk_path):
        fm.fontManager.addfont(cjk_path)
        prop = fm.FontProperties(fname=cjk_path)
        name = prop.get_name()
        plt.rcParams["font.family"] = name
```

```

plt.rcParams["font.sans-serif"] = [name]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["pdf.fonttype"] = 42          # TrueType embed
plt.rcParams["ps.fonttype"] = 42
plt.rcParams["axes.titlesize"] = 12
plt.rcParams["axes.labelsize"] = 11
plt.rcParams["xtick.labelsize"] = 9
plt.rcParams["ytick.labelsize"] = 9
plt.rcParams["legend.fontsize"] = 9
plt.rcParams["figure.dpi"] = 150
plt.rcParams["savefig.dpi"] = 300
plt.rcParams["savefig.bbox"] = "tight"

_setup_font()

# 配色（灰度可分，色盲友好近似）
PALETTE = ["#1f77b4", "#d62728", "#2ca02c", "#ff7f0e", "#9467bd",
            "#8c564b", "#e377c2", "#7f7f7f", "#bcbd22"]

# ----- 数据读取 -----
def load_summary():
    df = pd.read_csv(SUMMARY_CSV)
    # C1 缺失填 C2 (80PER_3_6C 策略无独立第一阶段)
    df["C1"] = df["C1"].fillna(df["C2"])
    return df

def load_cycles():
    df = pd.read_csv(CYCLE_CSV)
    return df

# ----- policy 解析 -----
def parse_policy(policy):
    """返回 (C1, Q1, C2), [] summary 一致校验。"""
    return None # 依赖 summary, 不解析

# ----- 寿命外推 -----
def fit_linear_life(cycles, soh_smooth, threshold=0.8):
    """线性 SOH = a + k*N (a=截距, k=斜率) 外推寿命。返回 (a, k, life)。"""
    x = np.asarray(cycles, float)
    y = np.asarray(soh_smooth, float)
    if len(x) < 3:
        return np.nan, np.nan, np.nan
    # np.polyfit degree=1 返回 [slope, intercept]
    k, a = np.polyfit(x, y, 1)
    if k >= 0: # 早期无衰减 (甚至上升), 无法外推寿命
        return a, k, np.nan
    life = (threshold - a) / k
    return a, k, life

def fit_exp_life(cycles, soh_smooth, threshold=0.8):
    """指数 SOH = exp(-lambda*N) 拟合 (log(SOH)=-lambda*N)。返回 (lambda, life)。
    更稳健: 用全部点, 约束 lambda<=0 则寿命 NaN。"""
    x = np.asarray(cycles, float)
    y = np.clip(np.asarray(soh_smooth, float), 1e-6, None)
    if len(x) < 3:
        return np.nan, np.nan
    ly = np.log(y)
    # y = -lambda*x + const, 斜率 = -lambda
    slope, _ = np.polyfit(x, ly, 1)
    lam = -slope
    if lam <= 0:

```

```

        return lam, np.nan
    life = -np.log(threshold) / lam
    return lam, life

def fit_power_life(cycles, soh_smooth, threshold=0.8):
    """SOH = 1 - alpha*N^beta 拟合。早期数据近平台，外推极不稳，仅作参考。"""
    x = np.asarray(cycles, float)
    y = np.asarray(soh_smooth, float)
    y = np.clip(y, 1e-6, None)
    diff = 1 - y
    mask = (x > 0) & (diff > 1e-9)
    if mask.sum() < 3:
        return np.nan, np.nan, np.nan
    lx = np.log(x[mask]); ly = np.log(diff[mask])
    b, la = np.polyfit(lx, ly, 1)
    alpha = np.exp(la)
    if b <= 0 or alpha <= 0:
        return alpha, b, np.nan
    life = ((1 - threshold) / alpha) ** (1 / b)
    return alpha, b, life

# ----- 输出辅助 -----
def save_json(obj, name):
    p = os.path.join(RESULTS_DIR, name)
    with open(p, "w", encoding="utf-8") as f:
        json.dump(obj, f, ensure_ascii=False, indent=2, default=str)
    return p

def fig_path(name):
    return os.path.join(FIG_DIR, name)

if __name__ == "__main__":
    s = load_summary()
    c = load_cycles()
    print("summary:", s.shape, "cycles:", c.shape)
    print("C1 null after fill:", s["C1"].isna().sum())
    print("policies:", s["policy"].nunique())
    print("test batteries:", int(s["prediction_test"].sum()))

```

problem1.py

```

"""
问题1：数据整理与快充策略对寿命影响初步分析。
- 整理电池-策略-寿命表
- SOH 衰减曲线（线性+规律外推寿命）
- 寿命分布统计
- 典型长/短寿命策略对比
"""

import os, json
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from utils import (load_summary, load_cycles, fit_linear_life, fit_power_life,
                    fit_exp_life, save_json, fig_path, PALETTE, RESULTS_DIR)

summ = load_summary()
cyc = load_cycles()

# ----- 构建每电池记录 -----
records = []
for bid, g in cyc.groupby("battery_id"):
    g = g.sort_values("cycle")
    N = g["cycle"].to_numpy(float)
    soh = g["SOH"].to_numpy(float)
    soh_s = g["SOH_smooth"].to_numpy(float)

```

```

meta = summ[summ["battery_id"] == bid].iloc[0]
a_lin, k_lin, life_lin = fit_linear_life(N, soh_s)
alpha_pw, beta_pw, life_pw = fit_power_life(N, soh_s)
lam_exp, life_exp = fit_exp_life(N, soh_s)
# 多模型寿命：用中位数综合（robust），线性+指数可用时取两者中位；[]律发散[]
cand = [v for v in [life_lin, life_exp] if np.isfinite(v) and 0 < v < 1e7]
life_robust = float(np.median(cand)) if cand else np.nan
rec = {
    "battery_id": int(bid),
    "policy": meta["policy"],
    "C1": float(meta["C1"]), "Q1": float(meta["Q1"]), "C2": float(meta["C2"]),
    "initial_capacity": float(meta["initial_capacity"]),
    "mean_chargetime": float(meta["mean_chargetime"]),
    "mean_IR": float(meta["mean_IR"]),
    "mean_Tavg": float(meta["mean_Tavg"]),
    "is_test": int(meta["prediction_test"]),
    "n_cycles": int(len(N)),
    "last_SOH": float(soh_s[-1]),
    "slope_SOH": float(k_lin), # per cycle
    "life_linear": float(life_lin),
    "life_exp": float(life_exp),
    "life_power": float(life_pw),
    "life": float(life_robust), # 主寿命代理（线性+指数中位）
    "IR_first": float(g["IR"].iloc[0]),
    "IR_last": float(g["IR"].iloc[-1]),
    "IR_growth_pct": float((g["IR"].iloc[-1] - g["IR"].iloc[0]) / g["IR"].iloc[0] *
100),
}
records.append(rec)
df1 = pd.DataFrame(records).sort_values("battery_id")
df1.to_csv(os.path.join(RESULTS_DIR, "p1_summary.csv"), index=False)

# 策略级寿命聚合（主寿命代理 life = 线性+指数寿命中位）
agg = df1.groupby("policy").agg(
    n=("battery_id", "count"),
    C1=("C1", "first"), Q1=("Q1", "first"), C2=("C2", "first"),
    life_linear_med=("life_linear", "median"),
    life_exp_med=("life_exp", "median"),
    life_med=("life", "median"),
    life_min=("life", "min"),
    life_max=("life", "max"),
    slope_med=("slope_SOH", "median"),
    mean_chargetime=("mean_chargetime", "mean"),
    mean_IR=("mean_IR", "mean"),
    mean_Tavg=("mean_Tavg", "mean"),
).reset_index().sort_values("life_med", ascending=False)
agg.to_csv(os.path.join(RESULTS_DIR, "p1_policy_life.csv"), index=False)

# 典型长/短寿命策略
long2 = agg.head(2)["policy"].tolist()
short2 = agg.tail(2)["policy"].tolist()
save_json({"long_life_top2": long2, "short_life_bottom2": short2,
    "policy_life": agg.to_dict(orient="records")}, "p1_typical.json")

print("=== 策略级寿命（按主寿命中位降序）===")
print(agg[["policy", "C1", "Q1", "C2", "n", "life_linear_med", "life_exp_med", "life_med", "slope_med", "mean_
print("\n长寿命策略:", long2)
print("短寿命策略:", short2)

# ===== 图1: 全[]池 SOH-循环曲线（按策略着色） =====
fig, ax = plt.subplots(figsize=(8, 5.2))

```

```

pols = sorted(cyc["policy"].unique())
for i, p in enumerate(pols):
    sub = cyc[cyc["policy"] == p]
    for bid, g in sub.groupby("battery_id"):
        g = g.sort_values("cycle")
        ax.plot(g["cycle"], g["SOH_smooth"], color=PALETTE[i % len(PALETTE)],
                alpha=0.55, lw=1.0)

# 0.8 阈值线
ax.axhline(0.8, color="k", ls="--", lw=1.0, alpha=0.7)
ax.text(2, 0.805, "寿命终止阈值 80% SOH", fontsize=8, color="k")
ax.set_xlabel("循环次数 N")
ax.set_ylabel("SOH (平滑)")
ax.set_xlim(0, 210)
ax.set_ylim(0.92, 1.01)

# 图例: 策略
from matplotlib.lines import Line2D
handles = [Line2D([0], [0], color=PALETTE[i % len(PALETTE)], lw=2, label=p) for i, p in
            enumerate(pols)]
ax.legend(handles=handles, loc="lower left", ncol=2, frameon=False, fontsize=7)
plt.tight_layout(); plt.savefig(fig_path("p1_soh_curves.pdf")); plt.close()

# ===== 图2: 各策略寿命分布箱线图 (按寿命中位排序) =====
order = agg["policy"].tolist()
fig, ax = plt.subplots(figsize=(9, 5))
data = [df1[df1["policy"] == p]["life"].dropna().values for p in order]
bp = ax.boxplot(data, vert=True, patch_artist=True, showmeans=True,
                meanprops=dict(marker="D", mfc="white", mec="k", ms=5))
for i, patch in enumerate(bp["boxes"]):
    patch.set_facecolor(PALETTE[i % len(PALETTE)]); patch.set_alpha(0.6)
ax.set_xticklabels(order, rotation=35, ha="right", fontsize=8)
ax.set_ylabel("循环寿命 (外推至 80% SOH)")
ax.set_yscale("log")
plt.tight_layout(); plt.savefig(fig_path("p1_life_box.pdf")); plt.close()

# ===== 图3: 策略参数 (C1,Q1,C2) 散点矩阵 vs 寿命 =====
fig, axes = plt.subplots(1, 3, figsize=(11, 3.6))
for ax, xcol, xlab in zip(axes, ["C1", "Q1", "C2"], ["第一阶段倍率 C1 (C)", "切换 SOC Q1 (%)", "第二阶段倍率 C2 (C)"]):
    for i, p in enumerate(pols):
        sub = df1[df1["policy"] == p]
        ax.scatter(sub[xcol], sub["life"], color=PALETTE[i % len(PALETTE)],
                  s=40, alpha=0.8, edgecolor="white", lw=0.5, label=p if ax is
axes[0] else None)
    ax.set_xlabel(xlab); ax.set_ylabel("循环寿命")
    ax.set_yscale("log")
axes[0].legend(fontsize=6, loc="upper right", frameon=False)
plt.tight_layout(); plt.savefig(fig_path("p1_param_life_scatter.pdf")); plt.close()

# ===== 图4: 长 vs 短寿命策略对比 =====
comp_metrics = ["life_med", "mean_chargetime", "mean_IR", "mean_Tavg", "slope_med"]
comp_labels = ["循环寿命", "平均充□□□(min)", "平均内阻(Ω)", "平均温度(℃)", "衰减斜率(×1e5)"]
comp_df = agg[agg["policy"].isin(long2 + short2)].copy()
comp_df["type"] = comp_df["policy"].apply(lambda p: "长寿命" if p in long2 else "短寿命")
# 归一化每个指标到 0-1 以画雷达
from math import pi
norm = comp_df.copy()
for m in comp_metrics:
    mn, mx = agg[m].min(), agg[m].max()
    # 寿命越大越好 -> 正向; 充□□□/内阻/温度/斜率越小越好 -> 反向
    if m == "life_med":
        norm[m] = (comp_df[m] - mn) / (mx - mn)

```

```

else:
    norm[m] = (mx - comp_df[m]) / (mx - mn)

fig = plt.figure(figsize=(8, 5.5))
ax = fig.add_subplot(111, polar=True)
cats = comp_labels
Ncat = len(cats)
angles = [n / float(Ncat) * 2 * pi for n in range(Ncat)]
angles += angles[:1]
for i, row in norm.iterrows():
    vals = row[comp_metrics].tolist(); vals += vals[:1]
    lab = f"{row['policy'][:18]}..({'长' if row['type']=='长寿命' else '短'})"
    ax.plot(angles, vals, lw=1.6, label=lab)
    ax.fill(angles, vals, alpha=0.08)
ax.set_xticks(angles[:-1])
ax.set_xticklabels(cats, fontsize=8)
ax.set_ylim(0, 1)
ax.legend(loc="upper right", bbox_to_anchor=(1.35, 1.1), fontsize=7, frameon=False)
plt.tight_layout(); plt.savefig(fig_path("p1_long_short_radar.pdf")); plt.close()

# ===== 图5: 充□□□ vs 寿命散点 (问题4 伏笔) =====
fig, ax = plt.subplots(figsize=(7, 4.8))
for i, p in enumerate(pols):
    sub = df1[df1["policy"] == p]
    ax.scatter(sub["mean_chargetime"], sub["life"], color=PALETTE[i % len(PALETTE)],
               s=50, alpha=0.8, edgecolor="white", lw=0.5, label=p)
ax.set_xlabel("平均充□□□ (min)")
ax.set_ylabel("循环寿命")
ax.set_yscale("log")
ax.legend(fontsize=6, ncol=2, loc="upper right", frameon=False)
plt.tight_layout(); plt.savefig(fig_path("p1_chargetime_life.pdf")); plt.close()

print("\n图已生成: p1_soh_curves.pdf, p1_life_box.pdf, p1_param_life_scatter.pdf,
p1_long_short_radar.pdf, p1_chargetime_life.pdf")

```

problem2.py

问题2：充□策略及其参数(C1,Q1,C2)□□池寿命衰减的影响分析。

三参数非完全独立□化 -> 分组对比 + Kruskal-Wallis + 置换检验 + 岭回归(含交互) + 方差分解 + SOC区间高倍率暴露。

```

import os, json
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.inspection import permutation_importance
from utils import (load_summary, load_cycles, save_json, fig_path, PALETTE,
RESULTS_DIR, SEED)

df1 = pd.read_csv(os.path.join(RESULTS_DIR, "p1_summary.csv"))
df1 = df1[np.isfinite(df1["life"])].reset_index(drop=True)

# 派生 SOC 区间高倍率暴露量
df1["E_low"] = df1["C1"] * df1["Q1"] # 低/中 SOC 段高倍率暴露 = C1 * Q1宽
df1["E_high"] = df1["C2"] * (80 - df1["Q1"]) # 中高 SOC 段高倍率暴露 = C2 * (80-Q1)宽
df1.to_csv(os.path.join(RESULTS_DIR, "p2_features.csv"), index=False)

# ===== (1) 策略间寿命差□□著性 =====
policies = sorted(df1["policy"].unique())
groups = [df1[df1["policy"] == p]["life"].values for p in policies]

```

```
# Kruskal-Wallis
H_stat, p_kw = stats.kruskal(*groups)
# 置换检验 (10000次): 在 H0 下随机重排策略标签, 看 H 统计量超过观测的比例
rng = np.random.RandomState(SEED)
life_arr = df1["life"].to_numpy()
pol_codes = df1["policy"].astype("category").cat.codes.to_numpy()
obs_H = H_stat
n_perm = 10000
perm_H = np.empty(n_perm)
for i in range(n_perm):
    perm = rng.permutation(pol_codes)
    g = [life_arr[perm == c] for c in range(len(policies))]
    if any(len(x) < 1 for x in g):
        perm_H[i] = 0.0
    else:
        perm_H[i] = stats.kruskal(*g).statistic
p_perm = np.mean(perm_H >= obs_H)

# 因素 ANOVA 对照 (非稳健, 小样本)
F_stat, p_anova = stats.f_oneway(*groups)

# 事后 Dunn (简单实现, Bonferroni)
from itertools import combinations
dunn = []
for a, b in combinations(range(len(policies)), 2):
    xa = groups[a]; xb = groups[b]
    U, p = stats.mannwhitneyu(xa, xb, alternative="two-sided")
    dunn.append({"pair": f"{policies[a]} vs {policies[b]}", "p": p,
                "diff_median": float(np.median(xa) - np.median(xb))})
dunn_df = pd.DataFrame(dunn)
dunn_df["p_bonf"] = (dunn_df["p"] * len(dunn_df)).clip(upper=1.0)
dunn_df.to_csv(os.path.join(RESULTS_DIR, "p2_dunn.csv"), index=False)

kw_result = {"kruskal_wallis_H": float(H_stat), "p_kw": float(p_kw),
             "p_permutation_10000": float(p_perm),
             "anova_F": float(F_stat), "p_anova": float(p_anova)}
save_json(kw_result, "p2_kw.json")
print("=== (1) 策略间寿命差异显著性 ===")
print(f"Kruskal-Wallis H={H_stat:.3f}, p={p_kw:.4g}; 置换p={p_perm:.4f}; ANOVA F={F_stat:.2f}, p={p_anova:.4g}")

# ===== (2) C1/Q1/C2 生命期望 + (3) 影响程度 =====
# 岭回归含交互 (多重共线性): life ~ C1 + Q1 + C2 + C1:Q1 + C2:Q1
X = df1[["C1", "Q1", "C2"]].copy()
X["C1_Q1"] = df1["C1"] * df1["Q1"]
X["C2_Q1"] = df1["C2"] * df1["Q1"]
y = np.log(df1["life"].values) # log 寿命, 稳定尺度
scaler = StandardScaler()
Xs = scaler.fit_transform(X)
ridge = Ridge(alpha=1.0)
ridge.fit(Xs, y)
yhat = ridge.predict(Xs)
r2_ridge = 1 - np.sum((y - yhat) ** 2) / np.sum((y - y.mean()) ** 2)
# 偏相关: 控制其他变量后因子 log(life) 的相关
def partial_corr(df_, target, var, controls):
    from sklearn.linear_model import LinearRegression
    Z = df_[controls].to_numpy()
    t = df_[target].to_numpy()
    v = df_[var].to_numpy()
    rt = t - LinearRegression().fit(Z, t).predict(Z)
```



```

    rv = v - LinearRegression().fit(Z, v).predict(Z)
    r, p = stats.pearsonr(rt, rv)
    return float(r), float(p)
pc = {}
for v, ctrl in [("C1", ["Q1", "C2"]), ("Q1", ["C1", "C2"]), ("C2", ["C1", "Q1"])]:
    tmp = df1.assign(log_life=y, C1_Q1=X["C1_Q1"], C2_Q1=X["C2_Q1"])
    r, p = partial_corr(tmp, "log_life", v, ctrl)
    pc[v] = {"partial_r": r, "p": p}

# 随机森林置换重要性（非参数影响程度）
rf = RandomForestRegressor(n_estimators=500, random_state=SEED)
Xrf = df1[["C1", "Q1", "C2"]].to_numpy()
rf.fit(Xrf, y)
perm_imp = permutation_importance(rf, Xrf, y, n_repeats=30, random_state=SEED)
imp = {k: float(np.mean(perm_imp.importances[i])) for i, k in enumerate(["C1", "Q1", "C2"])}
# 归一化为百分比
imp_pct = {k: v / sum(imp.values()) * 100 for k, v in imp.items()}

reg_result = {
    "ridge_r2": float(r2_ridge),
    "ridge_coefs_std": {k: float(c) for k, c in zip(X.columns, ridge.coef_)},
    "partial_corr": pc,
    "rf_perm_importance": imp,
    "rf_perm_importance_pct": imp_pct,
    "rf_r2": float(rf.score(Xrf, y)),
}
save_json(reg_result, "p2_regression.json")
print("\n=== (2/3) 岭回归 + 偏相 + 随机森林影响程度 ===")
print(f"岭回归 R²={r2_ridge:.3f} (log寿命, 标准化)")
print("偏相系数:", {k: f"{v['partial_r']:.3f} (p={v['p']:.3f})" for k, v in pc.items()})
print(f"随机森林 R²={rf.score(Xrf,y):.3f}, 置换重要性占比: {imp_pct}")

# ===== (4) SOC 区间高倍率衰减特征 =====
# 比较两组：高 E_high (中高SOC高倍率) vs 高 E_low (低SOC高倍率)，看衰减斜率 |slope|
# 用斜率绝对值对 E_low, E_high 做回归
from sklearn.linear_model import LinearRegression
sl = np.abs(df1["slope_S0H"].values) # 衰减速率（越大越快）
El = df1[["E_low", "E_high"]].to_numpy()
lr = LinearRegression().fit(El, sl)
sl_pred = lr.predict(El)
r2_sl = lr.score(El, sl)
# 偏相
r_low, p_low = stats.pearsonr(df1["E_low"], sl)
r_high, p_high = stats.pearsonr(df1["E_high"], sl)
soc_result = {
    "decay_vs_E_r2": float(r2_sl),
    "coefs": {"E_low": float(lr.coef_[0]), "E_high": float(lr.coef_[1]),
             "intercept": float(lr.intercept_)},
    "pearson_E_low_vs_slope": {"r": float(r_low), "p": float(p_low)},
    "pearson_E_high_vs_slope": {"r": float(r_high), "p": float(p_high)},
}
save_json(soc_result, "p2_soc_interval.json")
print("\n=== (4) SOC区间高倍率衰减特征 ===")
print(f"|斜率| ~ E_low+E_high: R²={r2_sl:.3f}, coef E_low={lr.coef_[0]:.2e}, E_high={lr.coef_[1]:.2e}")
print(f"Pearson: E_low vs |slope| r={r_low:.3f}(p={p_low:.3f}); E_high vs |slope| r={r_high:.3f}(p={p_high:.3f})")

# ===== 图 =====
# 图6：分组箱线图 + KW 显著性标注

```

```

fig, ax = plt.subplots(figsize=(9, 5))
order = df1.groupby("policy")
["life"].median().sort_values(ascending=False).index.tolist()
data = [df1[df1["policy"] == p]["life"].values for p in order]
bp = ax.boxplot(data, vert=True, patch_artist=True, showmeans=True,
                 meanprops=dict(marker="D", mfc="white", mec="k", ms=5))
for i, patch in enumerate(bp["boxes"]):
    patch.set_facecolor(PALETTE[i % len(PALETTE)]); patch.set_alpha(0.6)
ax.set_xticklabels(order, rotation=35, ha="right", fontsize=8)
ax.set_ylabel("循环寿命"); ax.set_yscale("log")
ax.set_title(f"Kruskal-Wallis H={H_stat:.1f}, p={p_kw:.3g} (置换p={p_perm:.3f})",
             fontsize=9)
plt.tight_layout(); plt.savefig(fig_path("p2_life_kw_box.pdf")); plt.close()

# 图7: C1/Q1/C2 vs 寿命 散点 + 岭回归拟合线
fig, axes = plt.subplots(1, 3, figsize=(11, 3.6))
for ax, col, lab in zip(axes, ["C1", "Q1", "C2"], ["C1 (C)", "Q1 (%)", "C2 (C)"]):
    for i, p in enumerate(policies):
        sub = df1[df1["policy"] == p]
        ax.scatter(sub[col], sub["life"], color=PALETTE[i % len(PALETTE)],
                  s=40, edgecolor="white", lw=0.5, alpha=0.85)
    ax.set_xlabel(lab); ax.set_ylabel("循环寿命"); ax.set_yscale("log")
# 偏相关注
r = pc[col]["partial_r"]; pp = pc[col]["p"]
ax.text(0.04, 0.04, f"偏相关 r={r:.2f}\np={pp:.3f}", transform=ax.transAxes,
        fontsize=7, va="bottom", ha="left")
plt.tight_layout(); plt.savefig(fig_path("p2_param_partial.pdf")); plt.close()

# 图8: 影响程度条形 (随机森林置换重要性 %)
fig, ax = plt.subplots(figsize=(6, 3.8))
keys = ["C1", "Q1", "C2"]
vals = [imp_pct[k] for k in keys]
bars = ax.bar(keys, vals, color=PALETTE[3], alpha=0.8, edgecolor="white")
ax.set_ylabel("命令方差解释占比 (%)")
for b, v in zip(bars, vals):
    ax.text(b.get_x() + b.get_width()/2, v + 0.5, f"{v:.1f}%", ha="center", fontsize=9)
plt.tight_layout(); plt.savefig(fig_path("p2_importance_bar.pdf")); plt.close()

# 图9: SOC 区间高倍率暴露 vs 衰减斜率
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, col, lab in zip(axes, ["E_low", "E_high"],
                        [f"低SOC段高倍率暴露 $E_{low}=C_{1Q_1}$", f"中高SOC段高倍率暴露 $E_{high}=C_2(80-Q_1)$"]):
    for i, p in enumerate(policies):
        sub = df1[df1["policy"] == p]
        ax.scatter(sub[col], sub["slope_SOH"].abs() * 1e5,
                  color=PALETTE[i % len(PALETTE)], s=40, edgecolor="white", lw=0.5,
                  label=p)
    ax.set_xlabel(lab); ax.set_ylabel("|衰减斜率| (×1e5)")
axes[0].legend(fontsize=6, ncol=1, loc="upper left", frameon=False)
plt.tight_layout(); plt.savefig(fig_path("p2_soc_exposure.pdf")); plt.close()

print("\n图已生成: p2_life_kw_box.pdf, p2_param_partial.pdf, p2_importance_bar.pdf, p2_soc_exposure.pdf")

```

问题3: 电池寿命预测。

- 防泄露: 40 块非测试电池做留出验证 (前N_train训练→151~200验证); 9 块测试电池仅前150可见, 预测151~200及寿命。
- 模型对比: 线性外推基线 / 指数 / 随机森林() / XGBoost()。
- 特征: 容量类+充+老化类+策略类+滑。

```

- 数据长度敏感性：前 50/100/150。
"""
import os, json, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from utils import (load_summary, load_cycles, fit_linear_life, fit_exp_life,
                    save_json, fig_path, PALETTE, RESULTS_DIR, SEED)

warnings.filterwarnings("ignore")
summ = load_summary()
cyc = load_cycles()

test_ids = set(summ[summ["prediction_test"] == 1]["battery_id"].astype(int))
all_ids = sorted(cyc["battery_id"].unique(), key=lambda x: int(x))
train_ids = [b for b in all_ids if int(b) not in test_ids] # 40 块
print(f"训练/验证电池(非测试): {len(train_ids)}; 测试电池: {len(test_ids)}")

# ----- 特征提取：□一块电池，从循环数据构造“到第N循环为止”的特征 -----
def features_upto(g, n_end, meta):
    """用前 n_end 个循环构造特征向量（不含未来信息）。□□段，g 可能含预测追加行。"""
    g = g.sort_values("cycle").head(n_end)
    N = g["cycle"].to_numpy(float)
    soh = g["SOH"].to_numpy(float)
    soh_s = g["SOH_smooth"].to_numpy(float)
    cap = g["capacity"].to_numpy(float)
    ir = g["IR"].to_numpy(float)
    tch = g["chargetime"].to_numpy(float)
    tavg = g["Tavg"].to_numpy(float)
    f = {}
    f["last_SOH"] = float(soh_s[-1])
    f["slope_SOH"] = float(np.polyfit(N, soh_s, 1)[0]) if len(N) >= 3 else 0.0
    f["curvature"] = float(np.polyfit(N, soh_s, 2)[0]) if len(N) >= 4 else 0.0
    f["last_cap"] = float(cap[-1])
    f["mean_IR"] = float(np.mean(ir))
    f["IR_growth"] = float((ir[-1] - ir[0]) / ir[0]) if ir[0] > 0 else 0.0
    f["mean_chargetime"] = float(np.mean(tch))
    f["chargetime_slope"] = float(np.polyfit(N, tch, 1)[0]) if len(N) >= 3 else 0.0
    f["mean_Tavg"] = float(np.mean(tavg))
    f["delta_cap"] = float(cap[0] - cap[-1])
    # 策略静态特征
    f["C1"] = float(meta["C1"]); f["Q1"] = float(meta["Q1"]); f["C2"] =
float(meta["C2"])
    f["E_low"] = f["C1"] * f["Q1"]
    f["E_high"] = f["C2"] * (80 - f["Q1"])
    return f

# ----- □□□□模型：预测 SOH 增量 -----
# 思路：用 40 块电池前 N_train 循环训练一个“下一循环 SOH”模型，
# 特征=当前状态特征(到第k循环)，目标=第k+1循环SOH_smooth。
# 预测时递归：从第N_train循环起，用预测值更新状态，逐步外推到200。
FEAT_KEYS = None # 占位，□行时填

def build_dataset(ids, n_train, target_horizon_start):
    """□每块电池，用前 n_train 循环构造训练样本：
    每个样本 = features_upto(k) -> SOH_smooth[k+1] (k 从 1 到 n_train-1)。"""
    X, y = [], []
    for bid in ids:
        g = cyc[cyc["battery_id"] == bid].sort_values("cycle")

```

```

    meta = summ[summ["battery_id"] == bid].iloc[0]
    soh_s = g["SOH_smooth"].to_numpy(float)
    for k in range(2, n_train): # 用前k个循环预测第k+1
        f = features_upto(g, k, meta)
        X.append(list(f.values()))
        y.append(soh_s[k]) # 第k+1循环 (0-index k)
    return np.array(X), np.array(y), list(f.keys()) if False else None

def get_feat_keys():
    g0 = cyc[cyc["battery_id"] == all_ids[0]].sort_values("cycle")
    m0 = summ[summ["battery_id"] == all_ids[0]].iloc[0]
    return list(features_upto(g0, 10, m0).keys())

FEAT_KEYS = get_feat_keys()
print("特征:", FEAT_KEYS)

# ----- 模型工厂 -----
def make_model(name):
    if name == "rf":
        return RandomForestRegressor(n_estimators=300, max_depth=8, random_state=SEED)
    return None

# ----- 留出验证: 40块, 前150训练->151..200验证 -----
N_TRAIN = 150
HORIZON = list(range(151, 201)) # 预测 151..200
X_tr, y_tr, _ = build_dataset(train_ids, N_TRAIN, 151)
scaler = StandardScaler().fit(X_tr)
X_tr_s = scaler.transform(X_tr)

# 基线模型: 纯线性外推 (每池独立, 不需训练集) — 直接每池前150拟合
def predict_linear_baseline(bid, n_train):
    g = cyc[cyc["battery_id"] == bid].sort_values("cycle")
    N = g["cycle"].to_numpy(float)[:n_train]
    soh_s = g["SOH_smooth"].to_numpy(float)[:n_train]
    slope, intercept = np.polyfit(N, soh_s, 1) # polyfit 返回 [斜率, 截距]
    return np.array([intercept + slope * n for n in HORIZON]), (intercept, slope)

def predict_exp_baseline(bid, n_train):
    g = cyc[cyc["battery_id"] == bid].sort_values("cycle")
    N = g["cycle"].to_numpy(float)[:n_train]
    soh_s = np.clip(g["SOH_smooth"].to_numpy(float)[:n_train], 1e-6, None)
    slope, const = np.polyfit(N, np.log(soh_s), 1) # log(SOH)=slope*N+const, slope=-
    lambda
    lam = -slope
    return np.array([np.exp(slope * n + const) for n in HORIZON]), lam

# RF 预测
def predict_recursive_ml(bid, n_train, model, scaler):
    g = cyc[cyc["battery_id"] == bid].sort_values("cycle")
    meta = summ[summ["battery_id"] == bid].iloc[0]
    # 用前 n_train 个循环的真实数据作起点
    hist = g.head(n_train).copy()
    preds = []
    cur_n = n_train
    # 构造一个可追加的 DataFrame 模拟未来循环
    for tgt_cycle in HORIZON:
        cur_n = tgt_cycle
        f = features_upto(hist, len(hist), meta)
        x = scaler.transform([list(f.values())])
        yhat = float(model.predict(x)[0])
        preds.append(yhat)

```

```

# 更新历史：追加一行（用预测 SOH，其余特征用最近一循环外推）
last = hist.iloc[-1].copy()
last["cycle"] = cur_n
last["SOH_smooth"] = yhat
last["SOH"] = yhat
# IR/chargeTime/Tavg 用线性外推
last["IR"] = hist["IR"].iloc[-1] # 近似持平
last["chargeTime"] = hist["chargeTime"].iloc[-1]
last["Tavg"] = hist["Tavg"].iloc[-1]
last["capacity"] = yhat * float(meta["initial_capacity"])
hist = pd.concat([hist, pd.DataFrame([last])], ignore_index=True)
return np.array(preds)

# 训练 RF
rf = make_model("rf"); rf.fit(X_tr_s, y_tr)

# 留出验证（40块，有真值）
metrics = {"linear": [], "exp": [], "rf": []}
preds_holdout = {}
for bid in train_ids:
    true = cyc[cyc["battery_id"] == bid].sort_values("cycle")
    ["SOH_smooth"].to_numpy(float)[150:200]
    if len(true) != 50:
        continue
    pl, _ = predict_linear_baseline(bid, 150)
    pe, _ = predict_exp_baseline(bid, 150)
    pr = predict_recursive_ml(bid, 150, rf, scaler)
    preds_holdout[bid] = {"true": true.tolist(), "linear": pl.tolist(),
                          "exp": pe.tolist(), "rf": pr.tolist()}
    for name, p in [("linear", pl), ("exp", pe), ("rf", pr)]:
        rmse = float(np.sqrt(mean_squared_error(true, p)))
        mae = float(mean_absolute_error(true, p))
        mape = float(np.mean(np.abs((true - p) / true)) * 100)
        metrics[name].append({"bid": int(bid), "rmse": rmse, "mae": mae, "mape": mape})

def agg_metrics(L):
    return {"rmse_mean": float(np.mean([m["rmse"] for m in L])),
            "rmse_med": float(np.median([m["rmse"] for m in L])),
            "mae_mean": float(np.mean([m["mae"] for m in L])),
            "mape_mean": float(np.mean([m["mape"] for m in L]))}

holdout_summary = {k: agg_metrics(v) for k, v in metrics.items()}
save_json({"holdout_summary_150train": holdout_summary,
          "per_battery": {k: v for k, v in [(b, metrics_col) for b, metrics_col in
[]]}}, "p3_holdout_metrics.json")
# 也存逐池
for name in ["linear", "exp", "rf"]:
    pd.DataFrame(metrics[name]).to_csv(os.path.join(RESULTS_DIR,
f"p3_holdout_{name}.csv"), index=False)

print("\n=== 留出验证（40块，前150训练→151~200验证） ===")
for k, v in holdout_summary.items():
    print(f"{k:7s}: RMSE_mean={v['rmse_mean']:.5f} (med={v['rmse_med']:.5f}),
MAE={v['mae_mean']:.5f}, MAPE={v['mape_mean']:.3f}%")

# 最优模型主模型
best_model_name = min(holdout_summary, key=lambda k: holdout_summary[k]["rmse_mean"])
print(f"\n最优模型: {best_model_name}")

# ----- 数据长度敏感性：前 50/100/150 -----
length_result = {}

```

```

for n_tr in [50, 100, 150]:
    Xn, yn, _ = build_dataset(train_ids, n_tr, n_tr + 1)
    scn = StandardScaler().fit(Xn); Xn_s = scn.transform(Xn)
    rfn = make_model("rf"); rfn.fit(Xn_s, yn)
    HORIZ = list(range(n_tr + 1, n_tr + 51)) # 预测后50循环
    rmses_lin, rmses_rf = [], []
    for bid in train_ids:
        g = cyc[cyc["battery_id"] == bid].sort_values("cycle")
        true = g["SOH_smooth"].to_numpy(float)[n_tr:n_tr + 50]
        if len(true) != 50:
            continue
        # 线性基线 (注意 polyfit 返回 [斜率, 截距])
        N = g["cycle"].to_numpy(float)[:n_tr]
        s = g["SOH_smooth"].to_numpy(float)[:n_tr]
        slope_lin, const_lin = np.polyfit(N, s, 1)
        pl = np.array([const_lin + slope_lin * n for n in HORIZ])
        # rf
        pr = predict_recursive_ml(bid, n_tr, rfn, scn)
        rmses_lin.append(float(np.sqrt(mean_squared_error(true, pl))))
        rmses_rf.append(float(np.sqrt(mean_squared_error(true, pr))))
    length_result[n_tr] = {"linear_rmse_mean": float(np.mean(rmses_lin)),
                           "rf_rmse_mean": float(np.mean(rmses_rf))}
save_json(length_result, "p3_data_length.json")
print("\n=== 数据长度敏感性 ===")
for n, v in length_result.items():
    print(f"前{n}循环: 线性RMSE={v['linear_rmse_mean']:.5f}, RF-
RMSE={v['rf_rmse_mean']:.5f}")

# ----- 9块测试电池预测 151~200 + 寿命 -----
# 用主模型 (按 holdout 最优); 同时给线性/指数照
pred_test_rows = []
life_test = []
fig, axes = plt.subplots(3, 3, figsize=(12, 9))
for ax, bid in zip(axes.ravel(), sorted(test_ids)):
    g = cyc[cyc["battery_id"] == bid].sort_values("cycle")
    meta = summ[summ["battery_id"] == bid].iloc[0]
    true150 = g["SOH_smooth"].to_numpy(float)[:150]
    # 主模型预测
    if best_model_name == "rf":
        pm = predict_recursive_ml(bid, 150, rf, scaler)
    elif best_model_name == "exp":
        pm, _ = predict_exp_baseline(bid, 150)
    else:
        pm, _ = predict_linear_baseline(bid, 150)
    pl, _ = predict_linear_baseline(bid, 150)
    pe, _ = predict_exp_baseline(bid, 150)
    # 寿命预测: 结合“前150斜率”“151..200预测”拟合直线外推。
    # 用 151..200 预测段拟合会因 50 点短而失稳 (斜率近 0 甚至正 -> NaN 或天文数字)。
    # 改: 前150 SOH_smooth + 151..200 预测 一起拟合一条直线, 更稳健地外推到 0.8。
    def life_from_combined(pred_seq):
        N_real = g["cycle"].to_numpy(float)[:150]
        S_real = g["SOH_smooth"].to_numpy(float)[:150]
        N_pred = np.arange(151, 201, dtype=float)
        Ns = np.concatenate([N_real, N_pred])
        Ss = np.concatenate([S_real, pred_seq])
        slope, intercept = np.polyfit(Ns, Ss, 1)
        if slope >= 0: return np.nan
        return (0.8 - intercept) / slope
    life_main = float(life_from_combined(pm))
    life_lin = float(life_from_combined(pl))
    life_exp = float(life_from_combined(pe))

```

```

pol = meta["policy"]
life_test.append({"battery_id": int(bid), "policy": pol,
                  "C1": float(meta["C1"]), "Q1": float(meta["Q1"]), "C2":
float(meta["C2"]),
                  "life_main": life_main, "life_linear": life_lin, "life_exp":
life_exp,
                  "model": best_model_name})
for i, (n, p) in enumerate([(151 + i, v) for i, v in enumerate(pm)]):
    pred_test_rows.append({"battery_id": int(bid), "cycle": n, "SOH_pred":
float(p),
                          "SOH_pred_linear": float(pl[i]),
                          "SOH_pred_exp": float(pe[i]),
                          "model": best_model_name})

# 画图：前150 + 预测
ax.plot(range(1, 151), true150, "k-", lw=1.2, label="实测(前150)")
ax.plot(range(151, 201), pm, "r-", lw=1.5, label=f"预测({best_model_name})")
ax.plot(range(151, 201), pl, "b--", lw=1.0, alpha=0.6, label="线性")
ax.plot(range(151, 201), pe, "g--", lw=1.0, alpha=0.6, label="指数")
ax.axhline(0.8, color="gray", ls=":", lw=0.8)
ax.set_title(f"# {bid} {pol[:18]}\n寿命预测={life_main:.0f}", fontsize=8)
ax.set_xlabel("循环"); ax.set_ylabel("SOH")
ax.legend(fontsize=6, loc="lower left"); ax.set_ylim(0.94, 1.005)
plt.tight_layout(); plt.savefig(fig_path("p3_test_pred_curves.pdf")); plt.close()

pd.DataFrame(pred_test_rows).to_csv(os.path.join(RESULTS_DIR, "p3_pred_test.csv"),
index=False)
pd.DataFrame(life_test).to_csv(os.path.join(RESULTS_DIR, "p3_life.csv"), index=False)
save_json({"best_model": best_model_name, "holdout": holdout_summary,
          "length_sensitivity": length_result, "test_life": life_test},
          "p3_summary.json")

print("\n=== 9块测试电池寿命预测 ===")
for r in life_test:
    print(f"#[{r['battery_id']}>2] {r['policy']}:38s life_main={r['life_main']:.0f}
(lin={r['life_linear']:.0f}, exp={r['life_exp']:.0f})")

# ----- 图：留出验证误差分布 -----
fig, ax = plt.subplots(figsize=(7, 4.5))
data = [pd.DataFrame(metrics[m])["rmse"].values for m in ["linear", "exp", "rf"]]
bp = ax.boxplot(data, tick_labels=["线性基线", "指数基线", "随机森林(□)"],
                patch_artist=True, showmeans=True,
                meanprops=dict(marker="D", mfc="white", mec="k", ms=5))
for i, p in enumerate(bp["boxes"]):
    p.set_facecolor(PALETTE[i]); p.set_alpha(0.6)
ax.set_ylabel("RMSE (151~200 循环 SOH) ")
for i, m in enumerate(["linear", "exp", "rf"]):
    ax.text(i + 1, holdout_summary[m]["rmse_mean"], f"均值={holdout_summary[m]
['rmse_mean']:.5f}",
            ha="center", va="bottom", fontsize=7)
plt.tight_layout(); plt.savefig(fig_path("p3_holdout_rmse_box.pdf")); plt.close()

# ----- 图：数据长度敏感性 -----
fig, ax = plt.subplots(figsize=(6, 4))
ns = sorted(length_result.keys())
ax.plot(ns, [length_result[n]["linear_rmse_mean"] for n in ns], "o-", label="线性基线")
ax.plot(ns, [length_result[n]["rf_rmse_mean"] for n in ns], "s-", label="随机森林(□)")
ax.set_xlabel("训练用早期循环数"); ax.set_ylabel("后50循环 RMSE (均值)")
ax.legend()
plt.tight_layout(); plt.savefig(fig_path("p3_data_length.pdf")); plt.close()

# ----- 图：特征重要性 (RF) -----

```



```

imp = dict(zip(FEAT_KEYS, rf.feature_importances_))
imp_s = dict(sorted(imp.items(), key=lambda x: -x[1]))
fig, ax = plt.subplots(figsize=(7, 4.5))
keys = list(imp_s.keys()); vals = list(imp_s.values())
bars = ax.barh(range(len(keys))[::-1], vals, color=PALETTE[0], alpha=0.8)
ax.set_yticks(range(len(keys))[::-1]); ax.set_yticklabels(keys, fontsize=8)
ax.set_xlabel("随机森林特征重要性")
plt.tight_layout(); plt.savefig(fig_path("p3_feature_importance.pdf")); plt.close()
save_json({"feature_importance": imp_s}, "p3_feature_importance.json")

print("\n图已生成: p3_test_pred_curves.pdf, p3_holdout_rmse_box.pdf, p3_data_length.pdf,
p3_feature_importance.pdf")
print("\n特征重要性:", imp_s)

```

problem4.py

问题4：兼顾充放电寿命衰减的充电策略优化。

- 充放电解析模型： $t_{ch} = t_1 + t_2 + t_3$, $t_1 = Q_1 / (100 * C_1) * 60$, $t_2 = (80 - Q_1) / (100 * C_2) * 60$, t_3 经验校准。

- SOH 衰减模型：复用问题2 的衰减斜率预测 $|slope| = f(E_{low}, E_{high}, C_1, Q_1, C_2)$ 。

- 寿命模型： $life = (0.8 - intercept) / slope$, $intercept \approx 1.0$ 初始。

- 多目标优化： $\min t_{ch}, \min |slope|$ ($\max life$)。先 9 策略离散比较，再邻域网格 + 加权 + Pareto。

- 固定种子，多起点。

```

"""
import os, json
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from utils import (load_summary, save_json, fig_path, PALETTE, RESULTS_DIR, SEED)

df1 = pd.read_csv(os.path.join(RESULTS_DIR, "p1_summary.csv"))
df1 = df1[np.isfinite(df1["life"])]
summ = load_summary()

# ===== (1) 充放电模型 =====
Q_rated = 1.1 # Ah
# t1, t2 解析 (小时)：恒流段  $I = C * Q_{rated}$ , 电量 =  $SOC * Q_{rated}$ ,  $t = \text{电量} / I = SOC / 100 / C$  (小时)
# 转 min: *60
# t3: CC-CV 段 (80%→满, 1C), 用数据校准:  $mean\_chargetime - (t_1 + t_2) * 60$ 
def t12_min(C1, Q1, C2):
    t1 = Q1 / 100.0 / C1 * 60.0 # min
    t2 = (80.0 - Q1) / 100.0 / C2 * 60.0
    return t1 + t2

# 校准 t3: 每块电池  $mean\_chargetime - t_{12}$ 
df1["t12_model"] = df1.apply(lambda r: t12_min(r["C1"], r["Q1"], r["C2"]), axis=1)
df1["t3_est"] = df1["mean_chargetime"] - df1["t12_model"]
t3_mean = float(df1["t3_est"].median()) # 用中位稳健
print(f"=== 充放电模型 ===")
print(f"t3 (CC-CV段) 中位校准 = {t3_mean:.3f} min")
print(f"t12 模型 vs 实测充电时间 R^2: ", end="")
r2_t = r2_score(df1["mean_chargetime"], df1["t12_model"] + t3_mean)
print(f"{r2_t:.3f}")
# 保存校准残差, 看是否策略相
df1[["battery_id", "policy", "C1", "Q1", "C2", "mean_chargetime", "t12_model",
"t3_est"]].to_csv(
    os.path.join(RESULTS_DIR, "p4_charge_time_calib.csv"), index=False)

def t_ch_model(C1, Q1, C2, t3=t3_mean):
    return t12_min(C1, Q1, C2) + t3

```



```
# ===== (2) SOH 衰减模型 =====
# 用[]连接保证预测的衰减斜率恒正 (物理约束: 倍率越高、中高SOC暴露越大 -> 衰减越快)。
#  $\log(|\text{slope}|) = b_0 + b_1 \cdot C1 + b_2 \cdot Q1 + b_3 \cdot C2 + b_4 \cdot E\_low + b_5 \cdot E\_high$ , 然后  $\exp$ 。
df1["E_low"] = df1["C1"] * df1["Q1"]
df1["E_high"] = df1["C2"] * (80 - df1["Q1"])
y_dec = np.abs(df1["slope_SOH"].to_numpy())
y_dec = np.clip(y_dec, 1e-12, None) # 防  $\log(0)$ 
log_y = np.log(y_dec)
X_dec = df1[["C1", "Q1", "C2", "E_low", "E_high"]].to_numpy()
dec_model = LinearRegression().fit(X_dec, log_y)
log_yhat = dec_model.predict(X_dec)
r2_dec = 1 - np.sum((log_y - log_yhat) ** 2) / np.sum((log_y - log_y.mean()) ** 2)
coefs = dict(zip(["C1", "Q1", "C2", "E_low", "E_high"], dec_model.coef_))
print(f"\n=== SOH 衰减模型 (连接, 恒正) ===")
print(f" $\log|\text{slope}| \sim C1+Q1+C2+E\_low+E\_high$ :  $R^2(\log\text{空间})=\{r2\_dec:.3f\}$ ")
print(f"系数: {coefs}, 截距={dec_model.intercept_:.3f}")

def slope_abs_pred(C1, Q1, C2):
    E_low = C1 * Q1; E_high = C2 * (80 - Q1)
    x = np.array([[C1, Q1, C2, E_low, E_high]])
    return float(np.exp(dec_model.predict(x)[0])) # 恒正

def life_pred(C1, Q1, C2, intercept0=1.0):
    s = slope_abs_pred(C1, Q1, C2) # 正的衰减速率
    # SOH = 1 - s*N -> 0.8 = 1 - s*L -> L = 0.2/s
    return 0.2 / s

# ===== (3) 多目标优化 =====
# (a) 9 种已有策略离散比较
strat = df1.groupby("policy").agg(
    C1=("C1", "first"), Q1=("Q1", "first"), C2=("C2", "first"),
    life_med=("life", "median"), mean_chargetime=("mean_chargetime", "mean"),
    slope_med=("slope_SOH", "median"),
).reset_index()
strat["t_ch_model"] = strat.apply(lambda r: t_ch_model(r["C1"], r["Q1"], r["C2"]),
axis=1)
strat["life_model"] = strat.apply(lambda r: life_pred(r["C1"], r["Q1"], r["C2"]),
axis=1)
strat["slope_model"] = strat.apply(lambda r: slope_abs_pred(r["C1"], r["Q1"], r["C2"]),
axis=1)
strat.to_csv(os.path.join(RESULTS_DIR, "p4_strategies.csv"), index=False)
print(f"\n=== 9 策略离散比较 (模型预测) ===")
print(strat[["policy", "C1", "Q1", "C2", "t_ch_model", "life_model",
"slope_model"]].to_string(index=False))

# (b) 邻域网格搜索 + 加权 + Pareto
# 无量纲化目标:  $t\_ch \square 1/\text{life}$  (或  $|\text{slope}|$ ) 都越小越好
# 先用 9 策略点计算 min/max 做归一化基准
t_min, t_max = strat["t_ch_model"].min(), strat["t_ch_model"].max()
s_min, s_max = strat["slope_model"].min(), strat["slope_model"].max()

def norm_t(t): return (t - t_min) / (t_max - t_min) if t_max > t_min else 0.0
def norm_s(s): return (s - s_min) / (s_max - s_min) if s_max > s_min else 0.0

def weighted_score(C1, Q1, C2, w_t=0.5, w_s=0.5):
    t = t_ch_model(C1, Q1, C2)
    s = slope_abs_pred(C1, Q1, C2)
    return w_t * norm_t(t) + w_s * norm_s(s)

# 网格: 在已有策略点  $\pm$  邻域内枚举
```

```

grid_pts = []
for _, r in strat.iterrows():
    for dc1 in [-0.5, -0.25, 0, 0.25, 0.5]:
        for dq in [-10, -5, 0, 5, 10]:
            for dc2 in [-0.5, -0.25, 0, 0.25, 0.5]:
                c1 = r["C1"] + dc1; q1 = r["Q1"] + dq; c2 = r["C2"] + dc2
                if c1 <= 0 or c2 <= 0 or q1 <= 0 or q1 >= 80: continue
                if c1 > 6 or c2 > 6: continue
                t = t_ch_model(c1, q1, c2); s = slope_abs_pred(c1, q1, c2); L =
life_pred(c1, q1, c2)
                grid_pts.append({"C1": c1, "Q1": q1, "C2": c2,
                                "near_policy": r["policy"],
                                "t_ch": t, "slope": s, "life": L,
                                "w_t0.5": weighted_score(c1, q1, c2, 0.5, 0.5)})
grid = pd.DataFrame(grid_pts).drop_duplicates(subset=["C1", "Q1", "C2"])
grid.to_csv(os.path.join(RESULTS_DIR, "p4_grid.csv"), index=False)

# Pareto 前沿 (min t_ch, min slope)
def is_pareto(df, cols):
    vals = df[cols].to_numpy()
    mask = np.ones(len(df), dtype=bool)
    for i in range(len(df)):
        for j in range(len(df)):
            if i == j: continue
            if np.all(vals[j] <= vals[i]) and np.any(vals[j] < vals[i]):
                mask[i] = False; break
    return mask
pareto_mask = is_pareto(grid, ["t_ch", "slope"])
grid["pareto"] = pareto_mask
pareto = grid[grid["pareto"]].sort_values("t_ch")
pareto.to_csv(os.path.join(RESULTS_DIR, "p4_pareto.csv"), index=False)

# 推荐策略：加权综合得分最低 (w_t=w_s=0.5)，且必须在 Pareto 前沿上
cand = grid[grid["pareto"]].copy()
best = cand.loc[cand["w_t0.5"].idxmin()]
rec = {"C1": float(best["C1"]), "Q1": float(best["Q1"]), "C2": float(best["C2"]),
        "near_policy": str(best["near_policy"]),
        "t_ch": float(best["t_ch"]), "life": float(best["life"]),
        "slope": float(best["slope"]),
        "weighted_score": float(best["w_t0.5"]),
        "t3_calibrated": t3_mean,
        "decay_model_r2_log": float(r2_dec),
        "charge_time_model_r2": float(r2_t)}
save_json(rec, "p4_recommendation.json")
print(f"\n=== 推荐策略 (加权 w_t=w_s=0.5, Pareto 上最优) ===")
print(json.dumps(rec, ensure_ascii=False, indent=2))

# 权重敏感性: w_t 从 0.2 到 0.8
sens = []
for w_t in np.arange(0.2, 0.81, 0.1):
    grid[f"w_{w_t:.1f}"] = grid.apply(lambda r: weighted_score(r["C1"], r["Q1"],
r["C2"], w_t, 1 - w_t), axis=1)
    b = grid.loc[grid[f"w_{w_t:.1f}"].idxmin()]
    sens.append({"w_t": float(w_t), "C1": float(b["C1"]), "Q1": float(b["Q1"]), "C2":
float(b["C2"]),
                "t_ch": float(b["t_ch"]), "life": float(b["life"])})
save_json(sens, "p4_weight_sensitivity.json")
print("\n=== 权重敏感性 ===")
for s in sens:
    print(f"w_t={s['w_t']:.1f}: C1={s['C1']:.2f} Q1={s['Q1']:.0f} C2={s['C2']:.2f}
t_ch={s['t_ch']:.2f} min life={s['life']:.0f}")

```

```
# ===== 图 =====
# 图10: Pareto 前沿 (充电 vs 寿命)
fig, ax = plt.subplots(figsize=(8, 5.5))
# 9 策略点
for i, p in enumerate(strat["policy"]):
    ax.scatter(strat[strat["policy"] == p]["t_ch_model"], strat[strat["policy"] == p]
["life_model"],
               color=PALETTE[i % len(PALETTE)], s=80, zorder=5, edgecolor="k", lw=0.6)
    r = strat[strat["policy"] == p].iloc[0]
    ax.annotate(p[14], (r["t_ch_model"], r["life_model"]), fontsize=6, xytext=(3, 3),
textcoords="offset points")
# Pareto 前沿
ax.plot(pareto["t_ch"], pareto["life"], "r-", lw=1.5, alpha=0.7, label="Pareto 前沿")
ax.scatter(pareto["t_ch"], pareto["life"], c="red", s=20, alpha=0.5, zorder=4)
# 推点
ax.scatter([best["t_ch"]], [best["life"]], marker="*", s=300, color="gold",
           edgecolor="k", lw=1.0, zorder=6, label=f"推策略
\n(C1={best['C1']:.2f}, Q1={best['Q1']:.0f}, C2={best['C2']:.2f})")
ax.set_xlabel("充电 (min)"); ax.set_ylabel("预测循环寿命"); ax.set_yscale("log")
ax.legend(loc="lower right", fontsize=8)
plt.tight_layout(); plt.savefig(fig_path("p4_pareto.pdf")); plt.close()

# 图11: 推策略在已有策略中的位置 (综合得分排名)
strat["score"] = strat.apply(lambda r: weighted_score(r["C1"], r["Q1"], r["C2"], 0.5,
0.5), axis=1)
strat = strat.sort_values("score")
fig, ax = plt.subplots(figsize=(9, 5))
colors = [PALETTE[i % len(PALETTE)] for i in range(len(strat))]
bars = ax.barh(range(len(strat)), strat["score"], color=colors, alpha=0.8)
ytick = [f"{p}\n(C1={row.C1:.1f}, Q1={row.Q1:.0f}, C2={row.C2:.1f})"
         for p, row in zip(strat["policy"], strat.itertuples())]
ax.set_yticks(range(len(strat))); ax.set_yticklabels(ytick, fontsize=7)
ax.set_xlabel("综合得分 (充电时间+衰减, 越小越好)")
ax.axvline(rec["weighted_score"], color="gold", ls="--", lw=1.2, label=f"推策略得分
={rec['weighted_score']:.3f}")
ax.legend(fontsize=8)
plt.tight_layout(); plt.savefig(fig_path("p4_score_rank.pdf")); plt.close()

# 图12: 充电模型拟合
fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(df1["mean_chargetime"], df1["t12_model"] + t3_mean, c=PALETTE[0], alpha=0.7,
s=40)
lims = [df1["mean_chargetime"].min(), df1["mean_chargetime"].max()]
ax.plot(lims, lims, "k--", lw=1, alpha=0.5)
ax.set_xlabel("实测平均充电时间 (min)"); ax.set_ylabel("解析模型预测 (min)")
ax.text(0.05, 0.95, f"R²={r2_t:.3f}", transform=ax.transAxes, va="top")
plt.tight_layout(); plt.savefig(fig_path("p4_chargetime_model.pdf")); plt.close()

# 图13: 权重敏感性
fig, ax = plt.subplots(figsize=(7, 4.5))
ws = [s["w_t"] for s in sens]
ax.plot(ws, [s["t_ch"] for s in sens], "o-", label="充电 (min)")
ax2 = ax.twinx()
ax2.plot(ws, [s["life"] for s in sens], "s-", color=PALETTE[1], label="预测寿命")
ax.set_xlabel("充电权重 w_t"); ax.set_ylabel("充电 (min)", color=PALETTE[0])
ax2.set_ylabel("预测寿命", color=PALETTE[1])
fig.legend(loc="upper center", ncol=2, fontsize=8)
plt.tight_layout(); plt.savefig(fig_path("p4_weight_sens.pdf")); plt.close()

print("\n图已生成: p4_pareto.pdf, p4_score_rank.pdf, p4_chargetime_model.pdf,
p4_weight_sens.pdf")
```